

Ordonnabilité des espaces métriques

Le sujet comporte 12 pages, numérotées de 1 à 12.

Début de l'épreuve.

Dans ce sujet, on s'intéresse au problème d'ordonner les éléments d'un espace métrique de sorte que deux éléments successifs soient à distance bornée.

Ce sujet est constitué de quatre parties. La première partie est formée de préliminaires utiles dans le reste du sujet, avec des questions de programmation en C et OCaml. La deuxième partie propose une implémentation en C d'une structure de données utilisée notamment dans la troisième partie. Cette troisième partie considère les graphes, munis de la distance de plus court chemin, comme espace métrique, avec implémentations en C. La quatrième partie, enfin, indépendante des deux précédentes, considère l'ordonnabilité des langages réguliers pour une certaine distance d'édition ; cette partie contient de la programmation en OCaml.

Dans les questions de programmation en C ou OCaml, on n'utilisera pas de fonctions qui ne sont pas incluses dans la bibliothèque standard du langage. Pour les codes en C, on pourra supposer que les en-têtes `<stdbool.h>`, `<stdio.h>`, `<stdlib.h>` et `<assert.h>` ont été inclus.

Espaces métriques et d -ordres

Dans ce sujet, on s'intéresse à des collections de données discrètes que l'on cherche à explorer ou engendrer de proche en proche. On modélise ce cadre général à l'aide des notions d'*espace métrique*, de *d -suite* et de *d -ordre* :

Définition 1 (espace métrique). *Un espace métrique $\mathcal{M} = (X, \delta)$ est constitué d'un ensemble dénombrable X dont les éléments sont appelés points et d'une distance sur X , c'est-à-dire une application $\delta : X \times X \rightarrow \mathbb{N}$ satisfaisant :*

Symétrie. Pour tout $(x, y) \in X^2$, $\delta(x, y) = \delta(y, x)$.

Séparation. Pour tout $(x, y) \in X^2$, $\delta(x, y) = 0$ si et seulement si $x = y$.

Inégalité triangulaire. Pour tout $(x, y, z) \in X^3$, $\delta(x, y) \leq \delta(x, z) + \delta(z, y)$.

Pour un espace métrique $\mathcal{M} = (X, \delta)$ et $X' \subseteq X$, on note $\mathcal{M}[X']$ le couple $(X', \delta|_{X'})$, où $\delta|_{X'}$ dénote la restriction de δ au domaine $X' \times X'$. On observe que $\mathcal{M}[X']$ est encore un espace métrique, que l'on appellera sous-espace de $\mathcal{M}$.

Définition 2 (d -suite). *Pour $d \in \mathbb{N}^*$, une d -suite $\mathbf{s}$ dans un espace métrique $\mathcal{M} = (X, \delta)$ est une suite (finie ou infinie dénombrable) $\mathbf{s} = x_1, x_2, \dots$ de points de $\mathcal{M}$ telle que :*

- $\mathbf{s}$ ne contient pas de doublons, c'est-à-dire que pour tous x_i, x_j de la suite, si $i \neq j$ alors $x_i \neq x_j$;
- deux points consécutifs de la suite sont à distance au plus d ; formellement, pour tous x_i, x_{i+1} de la suite on a $\delta(x_i, x_{i+1}) \leq d$.

On dit que $\mathbf{s}$ commence en x_1 et, dans le cas où $\mathbf{s}$ est finie et a n éléments, que $\mathbf{s}$ termine en x_n .

Définition 3 (d -ordre, d -ordonnable). *Pour $d \in \mathbb{N}^*$, un d -ordre de $\mathcal{M}$ est une d -suite dans $\mathcal{M}$ contenant tous les points de $\mathcal{M}$. L'espace $\mathcal{M}$ est dit d -ordonnable lorsqu'il existe un d -ordre de $\mathcal{M}$. On dit que $\mathcal{M}$ est ordonnable lorsque $\mathcal{M}$ est d -ordonnable pour un certain d .*

Distances d'édition sur les mots

On fixe dans tout le sujet l'*alphabet* $\Sigma := \{a, b\}$ ayant pour seules lettres a et b . Un *mot* w est une suite finie de lettres $w = \alpha_1 \cdots \alpha_n$. La longueur de w , notée $|w|$, est n . On note ε le mot vide, de longueur nulle. On note Σ^* l'ensemble des mots sur Σ . Un *langage* est un sous-ensemble de Σ^* .

Un espace métrique d'intérêt, sur l'ensemble des mots d'un langage, est donné par deux distances d'édition sur les mots : la distance *push-pop* et la distance *push-pop-droite*, que l'on définit ci-après.

Définition 4 (distance push-pop). *La distance push-pop, dénotée δ_{pp} , est définie de la manière suivante : pour $w, w' \in \Sigma^*$, $\delta_{pp}(w, w')$ est le nombre minimal d'opérations nécessaires pour passer de w à w' , où les opérations autorisées sont :*

- pour un mot w , insérer la lettre $\alpha \in \Sigma$ à la fin, ce qui donne le mot $w\alpha$;
- pour un mot w , insérer la lettre $\alpha \in \Sigma$ au début, ce qui donne le mot αw ;
- pour un mot de la forme $w\alpha$ avec $\alpha \in \Sigma$, supprimer la dernière lettre, ce qui donne le mot w ;
- pour un mot de la forme αw avec $\alpha \in \Sigma$, supprimer la première lettre, ce qui donne le mot w .

Exemple 1. *Les mots à distance push-pop 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin), $aabb$ (on a ajouté un b à la fin), ab (on a supprimé la première lettre), $aaab$ (on a ajouté un a au début) et $baab$ (on a ajouté un b au début).*

Définition 5 (distance push-pop-droite). *La distance push-pop-droite, dénotée δ_{ppr} , est définie de la même manière que δ_{pp} mais seules les insertions et suppressions à la fin du mot sont autorisées.*

Exemple 2. *Les mots qui sont à distance push-pop-droite 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin) et $aabb$ (on a ajouté un b à la fin).*

Algorithmes d'énumération push-pop et push-pop-droite

Lorsqu'on travaillera sur les mots, on considèrera parfois des programmes produisant une suite (potentiellement infinie) de mots de Σ^* , en utilisant les instructions spéciales suivantes : **popL()**, **popR()** ; **pushL(α)** et **pushR(α)** pour $\alpha \in \Sigma$; et **output()**.

Le programme dispose, comme état interne, d'une liste L d'éléments de Σ , qui est interprétée comme un mot de Σ^* . La liste L est initialement vide et représente donc le mot vide. Voici ce qu'il se passe lorsque le programme utilise les instructions spéciales :

- **popL()** a pour effet de supprimer la première lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- **popR()** a pour effet de supprimer la dernière lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- **pushL(α)** a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en début de liste ;
- **pushR(α)** a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en fin de liste ;
- **output()** *produit* le mot étant actuellement représenté par la liste (par exemple, il l'affiche en sortie du programme).

On souligne que la liste L n'est accessible par le programme que via ces instructions spéciales. De plus, on fait l'hypothèse que chacune des instructions **popL**, **popR**, **pushL** et **pushR** a une complexité en $O(1)$.

Un tel programme *produit* alors la suite de mots $w_1, w_2, \dots$, où w_1 est le premier mot produit par le programme lors de son exécution, w_2 le second et ainsi de suite.

On appellera un tel programme un *programme push-pop*. De manière similaire, un programme *push-pop-droite* est un programme push-pop qui n'utilise pas les instructions **popL()** ni **pushL(α)** pour $\alpha \in \Sigma$.

Exemple 3. On suppose que les fonctions `C` `pushL`, `pushR`, `popL`, `popR`, `output` ont été prédéfinies et manipulent toutes une même variable représentant la liste. Le programme `C` suivant est un programme push-pop-droite qui produit la suite de mots $w_1, w_2, \dots$ où w_i est $(aa)^{i-1}b$:

```
int main(void)
{
    pushR('b'); output();
    while (true) {
        popR(); pushR('a'); pushR('a'); pushR('b'); output();
    }
}
```

On remarque que ce programme ne termine jamais.

En OCaml, on suppose qu'on dispose d'un type `lettre` défini par :

```
type lettre = A | B
```

On représente alors les mots de Σ^* par des listes OCaml `lettre list`, dont la tête correspond à la première lettre du mot. Par exemple, le mot *aab* est représenté par la liste `[A;A;B]`. On suppose avoir accès en OCaml à des fonctions de type `pushR : lettre -> unit`, `pushL : lettre -> unit`, `popR : unit -> unit`, `popL : unit -> unit`, ainsi que `output : unit -> unit`, qui représentent les instructions spéciales.

Exemple 4. Le programme OCaml push-pop suivant produit la même suite que le programme `C` push-pop-droite de l'exemple 3 :

```
let main () =
    pushR B; output ();
    while true do
        pushL A; pushL A; output ();
    done
```

Partie I : Préliminaires

Question I.1. Soit $\mathcal{M} = (X, \delta)$ un espace métrique tel que X est un ensemble fini. Montrer que $\mathcal{M}$ est ordonnable.

Question I.2. Écrire une fonction OCaml

`delta_ppr : lettre list -> lettre list -> int`

prenant en entrée deux listes `l1`, `l2` représentant des mots w_1, w_2 et renvoyant $\delta_{\text{ppr}}(w_1, w_2)$, la distance push-pop-droite entre w_1 et w_2 . On attend de cette fonction que sa complexité soit linéaire en $|w_1| + |w_2|$.

Question I.3. On considère la suite $\mathbf{s} := w_1, w_2, \dots$, où w_i est a^{i^2} . Écrire un programme push-pop-droite en C qui produit la suite $\mathbf{s}$.

Question I.4. Montrer que $\mathcal{M}_{\text{pp}} := (\Sigma^*, \delta_{\text{pp}})$ et $\mathcal{M}_{\text{ppr}} := (\Sigma^*, \delta_{\text{ppr}})$ sont des espaces métriques.

On s'intéresse maintenant aux sous-espaces de $\mathcal{M}_{\text{pp}}$ et $\mathcal{M}_{\text{ppr}}$, c'est-à-dire (comme indiqué dans la Définition 1) aux espaces métriques de la forme $\mathcal{M}_{\text{pp}}[L] = (L, \delta_{\text{pp}|L})$ ou $\mathcal{M}_{\text{ppr}}[L] = (L, \delta_{\text{ppr}|L})$, pour L un langage. Par exemple, la suite produite par les programmes push-pop des exemples 3 et 4 est un 4-ordre pour $\mathcal{M}_{\text{ppr}}[L]$ et un 2-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := (aa)^*b$.

Question I.5. Écrire un programme push-pop en C qui produit un d -ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^* \mid b^*a^*$ et un certain $d \in \mathbb{N}$. Expliquer comment fonctionne ce programme.

Question I.6. Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^*$. [*Indice* : on peut visualiser le langage L comme la grille $\mathbb{N} \times \mathbb{N}$, où à la position (i, j) correspond le mot $a^i b^j$.] Ne pas hésiter à faire un dessin pour se faire comprendre.

Question I.7. Soit $L := b^* \mid ab^*$.

- Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$.
- Prouver que $\mathcal{M}_{\text{ppr}}[L]$ n'est pas ordonnable.

Question I.8. Un *chemin hamiltonien* dans un graphe non-orienté est un chemin (aussi appelé *chaîne*) qui visite tous les sommets du graphe exactement une fois. Un *graphe grille* est un graphe de la forme $G = (V, E_V)$ avec V une partie finie de $\mathbb{N} \times \mathbb{N}$ et $E_V = \{(i, j), (i', j')\} \mid (i, j), (i', j') \in V \text{ et } |i - i'| + |j - j'| = 1\}$, autrement dit, deux sommets sont connectés par une arête s'ils sont à distance 1 dans la grille $\mathbb{N} \times \mathbb{N}$.

On définit le problème de décision **HamGrille** de la manière suivante : en entrée on a un graphe grille G et la sortie est OUI si G possède un chemin hamiltonien et NON sinon. On admettra que ce problème est NP-complet.

Pour t un entier naturel non nul, on définit le problème de décision $\mathcal{P}_t$, de la façon suivante : $\mathcal{P}_t$ prend en entrée un ensemble fini de mots L et la sortie est OUI si $\mathcal{M}_{\text{pp}}[L]$ est t -ordonnable, NON sinon. On fixe maintenant un entier naturel $t \geq 1$ arbitraire. Montrer que pour cet entier t , le problème $\mathcal{P}_t$ est NP-complet.

Partie II : Implémentation en C des programmes push-pop

Dans cette partie, on souhaite réaliser une implémentation en langage C des opérations des programmes push-pop sur une structure de données `liste` codant une liste doublement chaînée. Le type de la structure de données `liste` est défini comme suit :

```
typedef struct chainon_s chainon;
```

```
struct chainon_s
{
    int val;
    chainon *prec;
    chainon *suiv;
};
```

```
struct liste_s
{
    chainon *premier;
    chainon *dernier;
};
```

```
typedef struct liste_s liste;
```

En particulier, les lettres sont donc représentées par des entiers (on rappelle qu'en C, une variable de type `char` peut être implicitement interprétée comme variable de type `int` sans perte d'informations); cela permettra de réutiliser cette structure `liste` dans d'autres contextes.

Dans l'implémentation de cette structure de données, on veillera à ce qu'un pointeur dont la valeur n'est pas `NULL` pointe toujours vers un espace mémoire valide et pas, par exemple, vers une donnée libérée.

Question II.1. Définir et initialiser une variable globale `lg` représentant la liste chaînée globale manipulée par les programmes push-pop; on souhaite que cette liste soit initialement vide.

Question II.2. Programmer une fonction `est_vide` de prototype `bool est_vide(void)` renvoyant `true` si `lg` représente une liste vide, `false` sinon.

Question II.3. Programmer des fonctions `pushL` et `pushR` de prototypes `void pushL(int)` et `void pushR(int)` implémentant les opérations `pushL` et `pushR` sur la liste représentée par `lg`. On utilisera une assertion pour vérifier que l'allocation dynamique de mémoire est bien réalisée. Quelle est la complexité en temps de ces deux fonctions ?

Question II.4. Programmer des fonctions `popL` et `popR` de prototypes `int popL(void)` et `int popR(void)` implémentant les opérations `popL` et `popR` sur la liste représentée par `lg` (chacune renvoyant également la valeur extraite de la liste). On utilisera une assertion pour s'assurer que la liste n'est pas vide et on veillera à libérer la mémoire devenue inutile. Quelle est la complexité en temps de ces deux fonctions ?

Question II.5. Programmer des fonctions `output` et `vide_liste` de prototypes respectifs `void output(void)` et `void vide_liste(void)`; `output` affiche le mot codé par la liste de caractères sur la sortie standard (suivi d'un retour à la ligne), tandis que `vide_liste` vide la liste (retire tous ses éléments) et libère la mémoire dynamique associée. Quelle est la complexité en temps de ces deux fonctions ?

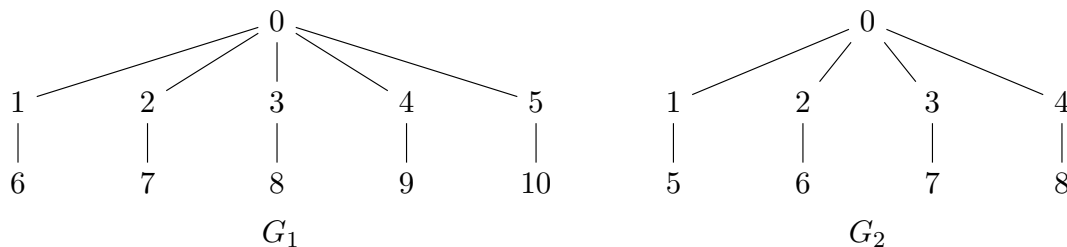
Partie III : Ordonnabilité des graphes

Dans cette partie on s'intéresse à des espaces métriques définis à partir de graphes non-orientés connexes. Pour un graphe non-orienté connexe $G = (V, E)$ (où V est l'ensemble fini non-vide de nœuds du graphe et E son ensemble d'arêtes), on note $\delta_G : V \times V \rightarrow \mathbb{N}$ la fonction qui à un couple de nœuds (u, v) de V associe la longueur d'un plus court chemin de u à v dans G .

L'objectif est de montrer que les espaces de la forme $\mathcal{M}_G := (V, \delta_G)$ sont 3-ordonnables et d'étudier des algorithmes qui calculent de tels ordres. On travaillera en langage C.

Question III.1. Soit $G = (V, E)$ un graphe non-orienté connexe. Montrer que le couple $\mathcal{M}_G := (V, \delta_G)$ est effectivement un espace métrique.

Question III.2. On considère les graphes G_1 et G_2 représentés ci-dessous.



- Donner un 3-ordre de $\mathcal{M}_{G_1}$ (aucune justification n'est attendue).
- Donner un 2-ordre de $\mathcal{M}_{G_2}$ (aucune justification n'est attendue).

On souhaite calculer un arbre couvrant d'un graphe non-orienté connexe. Un *arbre* est un graphe orienté *acyclique* $T = (V_T, E_T)$ avec V_T un ensemble fini non vide de nœuds et $E_T \subseteq V_T \times V_T$ un ensemble d'arcs tel que :

- il existe un unique nœud $r \in V_T$, appelé la *racine* de T tel que pour tout $v \in V_T$, $(v, r) \notin E_T$;
- pour tout nœud $u \in V_T$ qui n'est pas la racine, il existe un unique nœud $v \in V_T$ tel que $(v, u) \in E_T$ et on dit que u est un *fil* de v .

On définit maintenant un *arbre couvrant* d'un graphe non-orienté connexe $G = (V, E)$ comme un arbre $T = (V', E')$ tel que :

- $V' = V$;
- pour tout $(u, v) \in E'$, $\{u, v\} \in E$.

On va travailler avec la représentation en matrice d'adjacence des graphes : on suppose que le nombre de nœuds N est prédéfini en C comme une constante globale `N`, que les nœuds de G sont $\{0, \dots, N-1\}$ et que G est représenté par un tableau bidimensionnel `bool graphe[N][N]` où `g[u][v]` et `g[v][u]` valent tous les deux `true` si $\{u, v\}$ est une arête de G , et `false` sinon.

L'arbre couvrant T sera également représenté par un tableau `bool arbre[N][N]`, où cette fois `arbre[u][v]` vaut `true` si v est un fils de u et `false` sinon.

Question III.3. Écrire une fonction

```
void calcule_arbre_couvrant(bool graphe[N][N], bool arbre[N][N])
```

qui remplit la structure `arbre` pour que celle-ci représente un arbre couvrant quelconque du graphe G représenté par `graphe`, avec le nœud 0 comme racine. On supposera G connexe et on ne vérifiera pas ce point dans le code. On pourra utiliser les fonctions définies dans la partie II pour gérer une file (ou une pile) de nœuds à traiter. Justifier brièvement la correction de votre algorithme.

On supposera par la suite que `arbre` représente effectivement un arbre couvrant comme indiqué plus haut et que le nœud 0 est la racine de celui-ci.

Question III.4. On considère la fonction C suivante :

```
void visite(bool arbre[N][N], int noeud, bool mystere) {
    if(!mystere)
        pushR(noeud);
    for (int fils = 0; fils < N; ++fils) {
        if (arbre[noeud][fils])
            visite(arbre, fils, !mystere);
    }
    if(mystere)
        pushR(noeud);
}
```

On suppose que la liste `lg` utilisée par la fonctions `pushR` est initialement vide et qu'on appelle `visite(arbre, 0, false)` sur un arbre `arbre` de racine 0.

- Que vaut le troisième argument `mystere` lors d'un appel récursif à `visite` avec comme deuxième argument un nœud n de l'arbre, en fonction de la position de n dans l'arbre ?
- Que contient la liste `lg` à la fin de l'exécution de ce programme sur le graphe G_2 de la question III.2, vu comme un arbre enraciné en 0 ?

Question III.5. Montrer que l'appel `visite(a, 0, false)`, lorsque a est la représentation d'un arbre couvrant d'un graphe non-orienté connexe G de racine 0, calcule (dans la liste `lg`) un 3-ordre pour $\mathcal{M}_G$.

Question III.6. Quelle est la complexité en temps de l'appel à `visite(a, 0, false)`, en fonction du nombre N de nœuds ? Peut-on améliorer cette complexité (par exemple en changeant notre méthode de représentation des graphes) ?

Question III.7. On observe que la fonction `visite` est une fonction récursive.

- Quel(s) problème(s) cela pourrait-il poser ?
- Proposer une méthode pour rendre ce calcul non récursif. On ne s'attend pas ici à du code, mais à une explication qui puisse être facilement transformée en du code.

Partie IV : Ordonnabilité des langages réguliers pour la distance push-pop-droite

Dans cette partie on souhaite caractériser les langages réguliers ordonnables pour la distance push-pop-droite. On travaillera en OCaml.

Automates et langages réguliers. Le terme *automate* désignera systématiquement un automate fini déterministe. Formellement, un automate $A = (Q, q_{\text{init}}, F, \text{trans})$ consiste en un ensemble fini Q d'états, un état initial q_{init} , un ensemble $F \subseteq Q$ d'états finaux, ainsi qu'une fonction de transition $\text{trans} : Q \times \Sigma \rightarrow Q \cup \{\perp\}$, où $\perp$ signifie que la transition n'est pas définie. Un chemin dans A d'un état $q \in Q$ à un état $q' \in Q$ est une suite finie de la forme $q_1, \alpha_1, q_2, \alpha_2, \dots, \alpha_{n-1}, q_n$ où les q_i sont des états de Q et les α_i des lettres de Σ , telle que $q = q_1$, $q' = q_n$ et telle que pour tout $i \in \{1, \dots, n-1\}$ on a $q_{i+1} = \text{trans}(q_i, \alpha_i)$; l'étiquette d'un tel chemin est alors le mot $\alpha_1 \dots \alpha_{n-1}$. En particulier, il y a toujours un chemin de longueur nulle, avec étiquette ε , entre n'importe quel état et lui-même (la suite comprenant ce seul état). Le langage accepté par A , noté $L(A)$, est l'ensemble des mots qui étiquettent un chemin depuis q_{init} jusqu'à un état final. Un langage est *régulier* s'il est accepté par un automate ou, de manière équivalente, s'il est représenté par une expression régulière.

On rappelle qu'un automate $A = (Q, q_{\text{init}}, F, \text{trans})$ est *émondé* si tout état q est à la fois accessible depuis l'état initial (c'est-à-dire qu'il y a un chemin depuis q_{init} à q) et co-accessible depuis un état final (c'est-à-dire qu'il y a un chemin depuis q vers un état final). Pour tout automate A , on peut calculer en temps linéaire un automate A' émondé tel que $L(A) = L(A')$.

Automates poêle. D'après la question I.1, si L est fini alors $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable. **On supposera L infini dans cette partie.** On définit dans ce qui suit la notion d'*automate poêle-à-frîre* (ou *automate poêle*), puis on montre que $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable si et seulement si n'importe quel automate émondé acceptant L est un automate poêle.

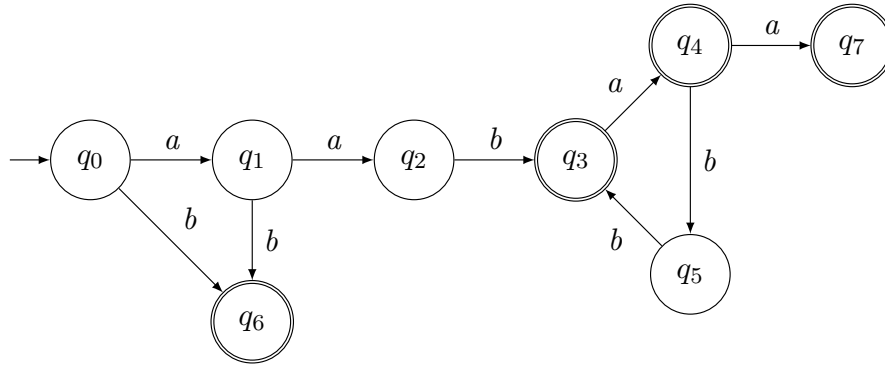
Soit $A = (Q, q_{\text{init}}, F, \text{trans})$ un automate. Un *cycle* dans A est un chemin de longueur non nulle dans A depuis un état q vers lui-même sans répétition d'état (à part q). On observe que ce chemin est possiblement de longueur 1, dans le cas où $\text{trans}(q, \alpha) = q$ pour $\alpha \in \Sigma$. On note que si $q_1, \alpha_1, q_2, \alpha_2, \dots, \alpha_{n-1}, q_1$ est un cycle, alors $q_2, \alpha_2, \dots, \alpha_{n-1}, q_1, \alpha_1, q_2$ et plus généralement $q_i, \alpha_i, \dots, \alpha_{n-1}, q_1, \alpha_1, \dots, \alpha_{i-1}, q_i$ pour tout $2 \leq i \leq n-2$ sont également des cycles : on dit que ce sont les mêmes cycles à *permutation près*.

Un *chemin strict vers un cycle* dans A est un chemin sans répétition d'état depuis l'état initial jusqu'à un état faisant partie d'un cycle tel que tous les états sauf le dernier ne font pas partie d'un cycle dans A ; formellement, c'est un chemin $q_1, \alpha_1, \dots, \alpha_{n-1}, q_n$ pour $n \in \mathbb{N}$ avec $q_1 = q_{\text{init}}$ tel que q_n fait partie d'un cycle et, pour $1 \leq i < n$, q_i ne fait partie d'aucun cycle. En particulier, si q_{init} fait partie d'un cycle alors le seul tel chemin est de longueur nulle.

On dit de A qu'il est *pseudo-acyclique* s'il a au plus un cycle à permutation près. On note qu'un automate tel que $\text{trans}(q, a) = \text{trans}(q, b) = q$ n'est jamais pseudo-acyclique, car q, a, q et q, b, q sont deux cycles différents.

Un automate A est alors un *automate poêle* s'il est pseudo-acyclique et a un unique chemin strict vers un cycle.

Exemple 5. *Considérons l'automate ci-dessous :*



Son ensemble d'états est $\{q_0, \dots, q_7\}$, son état initial q_0 , ses états finaux q_3, q_4, q_6 et q_7 ; les transitions non spécifiées par des flèches sont non définies. Cet automate est clairement déterministe et il est aisé de vérifier qu'il est émondé.

L'automate est également pseudo-acyclique : il comporte en effet un unique cycle, le cycle $q_3, a, q_4, b, q_5, b, q_3$ (qu'on peut également écrire $q_4, b, q_5, b, q_3, a, q_4$ ou encore $q_5, b, q_3, a, q_4, b, q_5$). Et comme il a un unique chemin strict vers un cycle (le chemin $q_0, a, q_1, a, q_2, b, q_3$), c'est un automate poêle.

Si une transition de q_3 à q_4 étiquetée par b était ajoutée, ce ne serait plus un automate pseudo-acyclique car $q_3, a, q_4, b, q_5, b, q_3$ et $q_3, b, q_4, b, q_5, b, q_3$ seraient deux cycles différents. De même, si une transition étiquetée par a de q_2 à l'un des états du cycle était ajoutée, l'automate aurait deux chemins stricts distincts vers un cycle et ne serait donc plus un automate poêle.

Question IV.1. Proposer une méthode permettant de déterminer si un automate émondé est un automate poêle, en supposant n'importe quelle représentation raisonnable des automates (on supposera l'automate émondé). On ne demande pas du code, mais on attend une explication qui puisse être facilement transformée en du code.

Lorsque A est un automate poêle, on appelle *état d'entrée* l'état qui se trouve à la fin de l'unique chemin strict vers l'unique cycle de A (cet état peut être l'état initial si celui-ci fait partie du cycle). Dans l'exemple 5, l'état q_3 est l'état d'entrée.

Question IV.2. Si A est un automate poêle, montrer que $L(A)$ peut s'écrire de la forme

$$F \mid uv^*F'$$

où F et F' sont des ensembles finis de mots et u, v sont des mots avec $v \neq \varepsilon$.

Question IV.3. On suppose que A est un automate poêle. Écrire un programme push-pop-droite en OCaml qui calcule un d -ordre pour $\mathcal{M}_{\text{ppr}}[L(A)]$, pour un certain $d \in \mathbb{N}$ que l'on ne cherchera pas à calculer, étant donnés les mots u, v et ensembles F, F' de la question précédente représentés en OCaml par des variables :

```

u : lettre list
v : lettre list
f : (lettre list) list
f' : (lettre list) list

```

On a ainsi établi que lorsqu'un langage L est accepté par un automate poêle alors il est ordonnable pour la distance push-pop-droite. On montre dans la suite de cette partie que c'est en fait une condition nécessaire, dans le sens où si $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable, alors n'importe quel automate émondé pour L est un automate poêle.

Pour ce faire, on considère l'arbre infini T_Σ de Σ^* défini ainsi : les nœuds de T_Σ sont les mots de Σ^* , le mot vide est la racine de T_Σ et si $w \in \Sigma^*$ alors wa et wb sont les fils de w dans T_Σ . Une *branche infinie* B dans T_Σ est une suite infinie de la forme $w_1, w_2, \dots$, où $w_1 = \varepsilon$ et $w_{i+1} = w_i \alpha_i$ avec $\alpha_i \in \Sigma$ pour tout $i \in \mathbb{N}$. Pour $w \in \Sigma^*$, on note T_w le sous-arbre infini de T_Σ enraciné en w .

Pour un langage L , une *branche lourde* est une branche infinie $B = w_1, w_2, \dots$ dans T_Σ telle que pour tout $i \in \mathbb{N}$, T_{w_i} contient une infinité de mots de L .

Question IV.4. Montrer que, pour n'importe quel langage L , si L est infini alors L a (au moins) une branche lourde.

Question IV.5. Montrer que, pour n'importe quel langage L , si $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable alors L a au plus une branche lourde.

Question IV.6. Soit A un automate émondé qui a au moins deux chemins stricts (distincts) vers des cycles (potentiellement identiques) et soient u et u' des mots étiquetant deux tels chemins.

- Montrer que u n'est pas un préfixe de u' et que u' n'est pas un préfixe de u .
- Montrer que $L(A)$ a au moins deux branches lourdes.

Question IV.7. Soit A un automate émondé qui a un seul chemin strict vers un cycle. Montrer que si A n'est pas pseudo-acyclique alors $L(A)$ a au moins deux branches lourdes. En déduire que pour un langage régulier L infini, $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable si et seulement si n'importe quel automate émondé A acceptant L est un automate poêle.

Fin du sujet.

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2023

**MARDI 18 AVRIL 2023
14h00 - 18h00**

**FILIERE MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la
composition de Physique et Sciences de l'Ingénieur.
Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Compression entropique

Le sujet comporte 11 pages, numérotées de 1 à 11.

Vue d'ensemble du sujet.

Ce sujet s'intéresse à la compression et à la décompression de mots sur un alphabet fini $\mathcal{S}$ de cardinal $|\mathcal{S}| \geq 2$. Plus précisément, on se donne une fonction q de $\mathcal{S}$ dans $\mathbb{N}$ et on s'intéresse aux mots de $\mathcal{S}^*$ tels que chaque lettre $\sigma \in \mathcal{S}$ a exactement $q(\sigma)$ occurrences dans ces mots. On note $\mathcal{S}^q$ l'ensemble de ces mots. Remarque : les mots de $\mathcal{S}^q$ ont pour longueur $N = \sum_{\sigma \in \mathcal{S}} q(\sigma)$.

Une lettre de $\mathcal{S}$ peut être représentée avec $\lceil \log_2 |\mathcal{S}| \rceil$ bits, où $\lceil x \rceil$ désigne la partie entière supérieure de x , c'est-à-dire l'entier tel que $\lceil x \rceil - 1 < x \leq \lceil x \rceil$. Si l'on concatène les bits représentant chacune des lettres d'un mot de $\mathcal{S}^q$, on peut donc représenter ce mot de façon non ambiguë à l'aide d'une séquence de bits de taille

$$\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \lceil \log_2 |\mathcal{S}| \rceil = \lceil \log_2 |\mathcal{S}| \rceil \cdot N.$$

Il s'agit d'une représentation non compressée.

La théorie de l'information affirme qu'il faut au moins $\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \log_2(N/q(\sigma))$ bits pour représenter tous les mots de $\mathcal{S}^q$ de façon non ambiguë. Ce sujet explore quelques façons de compresser les mots pour s'approcher de cette borne théorique.

La partie I s'intéresse à la fonction q , c'est-à-dire au comptage des lettres d'un mot. La partie II étudie les arbres binaires dont les feuilles sont étiquetées par des lettres de $\mathcal{S}$. La partie III utilise ces arbres pour (dé)compresser des mots de $\mathcal{S}^*$. La partie IV s'intéresse aux arbres qui donnent les meilleurs taux de compression pour les mots de $\mathcal{S}^q$. Certains de ces arbres ont une représentation compacte ; c'est l'objet de la partie V. D'autres arbres sont encore plus compacts, mais au prix d'un moindre taux de compression, comme le montre la partie VI. Finalement, la partie VII attaque le problème d'une façon complètement différente afin de s'approcher un peu plus du taux de compression théorique optimal.

Les différentes parties sont indépendantes : il n'est pas nécessaire d'avoir répondu aux questions d'une partie pour répondre aux questions d'une autre partie ; cependant les notions abordées dans une partie sont souvent utiles aux parties suivantes.

Rappels d'OCaml

On rappelle ici quelques opérations de base sur les tableaux :

- `Array.length t` renvoie la longueur du tableau `t`.
- `Array.make n v` crée un tableau de `n` cases qui sont toutes initialisées avec `v`.
- `Array.of_list l` renvoie un tableau initialisé avec les valeurs de la liste `l`, tandis que `Array.to_list t` réalise l'opération inverse.
- La case numéro `i` du tableau `t` peut être accédée avec `t.(i)`. Les cases sont numérotées à partir de zéro.

Partie I. Comptage d'occurrences

Un texte non compressé est un mot de $\mathcal{S}^*$. Dans la suite, à chaque fois qu'il s'agira de définir une fonction en OCaml, les lettres de $\mathcal{S}$ seront représentées par des entiers et un mot de $\mathcal{S}^*$ sera représenté par un tableau d'entiers (`int array`) ou une liste d'entiers (`int list`) en fonction des questions. La première étape consiste à calculer le nombre d'occurrences $q(\sigma)$ de chaque lettre σ dans un mot.

Question I.1. Supposons $\mathcal{S} = [0, 255]$. Définissez une fonction OCaml `occurrences : int list -> int array` qui reçoit en argument un mot représenté par une liste d'entiers et renvoie le nombre d'occurrences $q(\sigma)$ de chaque lettre $\sigma \in \mathcal{S}$ sous forme d'un tableau $\llbracket q(0); q(1); \dots; q(255) \rrbracket$.

Il peut être intéressant de ne considérer que le sous-ensemble des lettres qui ont au moins une occurrence. Dans la question suivante, on ne représentera pas q par $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$ mais par un tableau de paires $\llbracket (\sigma_1, q(\sigma_1)); (\sigma_2, q(\sigma_2)); \dots \rrbracket$ avec $\{\sigma_1, \sigma_2, \dots\} = \{\sigma \in \mathcal{S} \mid q(\sigma) \neq 0\}$ et $\sigma_1 < \sigma_2 < \dots$.

Question I.2. Définissez une fonction OCaml `nonzero_occurrences : int array -> (int * int) array` qui passe de la représentation de q de la question I.1 (tableau $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$) à la représentation sous forme d'un tableau de paires. Cette fonction devra avoir une complexité temporelle en $O(|\mathcal{S}|)$.

Partie II. Arbres binaires

Soit $\mathcal{T}_{\mathcal{S}}$ l'ensemble des arbres binaires dont les feuilles sont en bijection avec un ensemble fini $\mathcal{S}$. Autrement dit, un arbre de $\mathcal{T}_{\mathcal{S}}$ a exactement $|\mathcal{S}|$ feuilles ; chacune de ses feuilles est étiquetée par un élément de $\mathcal{S}$; toutes les feuilles sont étiquetées par des éléments différents de $\mathcal{S}$. Ces arbres peuvent être construits comme suit :

- Une feuille étiquetée par x est notée $F(x)$. C'est un arbre de $\mathcal{T}_{\{x\}}$.
- Si g et d sont des arbres appartenant respectivement à $\mathcal{T}_{\mathcal{S}_1}$ et $\mathcal{T}_{\mathcal{S}_2}$, alors l'arbre dont le sous-arbre gauche est g et le sous-arbre droit est d , noté $N(g, d)$, est un arbre de $\mathcal{T}_{\mathcal{S}_1 \cup \mathcal{S}_2}$ à condition que $\mathcal{S}_1 \cap \mathcal{S}_2 = \emptyset$.

Remarque : un tel arbre binaire possède exactement $|\mathcal{S}| - 1$ nœuds internes. Par ailleurs, on étiquettera implicitement les arêtes par 0 ou 1 suivant qu'elles mènent à un sous-arbre gauche (0) ou droit (1).

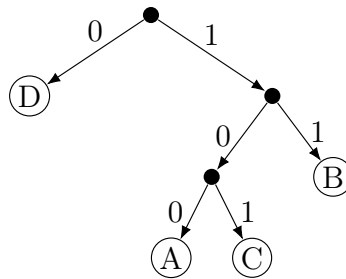


FIGURE 1 – Exemple d'arbre de $\mathcal{T}_{\{A,B,C,D\}}$.

Question II.1. Montrez que l'ensemble $\mathcal{T}_{\{A,B,C,D\}}$ contient 120 arbres.

Le type OCaml utilisé pour représenter les arbres est le suivant :

```
type tree = F of int | N of tree * tree
```

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$ et un élément σ de $\mathcal{S}$, on note $\ell_t(\sigma)$ la profondeur de la feuille étiquetée par σ , c'est-à-dire le nombre d'arêtes entre la racine de t et cette feuille. Par exemple, avec l'arbre de la figure 1, on a $\ell_t(A) = 3$.

Comme précédemment, soit q une fonction de $\mathcal{S}$ dans $\mathbb{N}$. Pour tout arbre $t \in \mathcal{T}_{\mathcal{S}'}$ avec $\mathcal{S}' \subseteq \mathcal{S}$, on note $c_q(t)$ la somme pondérée suivante :

$$c_q(t) = \sum_{\sigma \in \mathcal{S}'} q(\sigma) \ell_t(\sigma).$$

Question II.2. Supposons $\mathcal{S} = [0, n-1]$. Définissez une fonction OCaml `cq : tree -> int array -> int` qui reçoit deux arguments, un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et un tableau $\llbracket q(0); q(1); \dots; q(n-1) \rrbracket$ représentant q , et qui renvoie la valeur de $c_q(t)$. On cherchera à écrire une fonction efficace. Donnez et justifiez sa complexité temporelle.

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$, on représente une lettre $\sigma \in \mathcal{S}$ par la séquence des bits obtenus en parcourant t de la racine vers la feuille $F(\sigma)$. Par exemple, dans l'arbre de $\mathcal{T}_{\{A,B,C,D\}}$ de la figure 1, D est représenté par 0 tandis que A est représenté par 100.

Question II.3. Définissez une fonction OCaml `get_path : int -> tree -> int list` qui reçoit en argument un entier $\sigma \in \mathcal{S}$ et un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste de 0 et 1 représentant σ dans t . Donnez et justifiez la complexité temporelle de cette fonction.

Considérons les séquences de bits définies de la façon suivante. Elles commencent par $k \geq 0$ bits valant 1, suivis d'un bit à 0, suivis de k bits arbitraires, ce que l'on notera $1^k 0(0 \mid 1)^k$. Les dix premières séquences par ordre lexicographique sont donc 0, 100, 101, 11000, 11001, 11010, 11011, 1110000, 1110001, 1110010. L'arbre dont les branches constituent ces séquences et dont les feuilles sont étiquetées de gauche à droite par des entiers croissants commence comme illustré sur la figure 2.

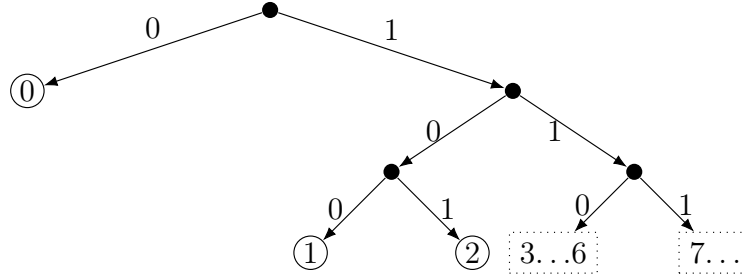


FIGURE 2 – Arbre des séquences $1^k 0(0 \mid 1)^k$.

Remarque : l'étiquette de chaque feuille correspond à l'indice de la séquence quand elles sont triées par ordre lexicographique. Les séquences croissant indéfiniment, l'arbre est *a priori* infini. C'est pourquoi la question suivante se limite aux séquences de bits $1^k 0(0 \mid 1)^k$ pour lesquelles k n'excède pas une certaine borne ℓ , afin d'obtenir un arbre de profondeur bornée $2\ell + 1$.

Question II.4. Définissez une fonction OCaml `integers : int -> tree` qui prend un entier $\ell \geq 0$ en argument et renvoie l'arbre dont les branches sont les séquences de la forme $1^k 0(0 \mid 1)^k$ avec $k \leq \ell$, ainsi que la séquence $1^{\ell+1}$. Les feuilles seront étiquetées de gauche à droite par les entiers 0, 1, 2, etc.

Partie III. Codes préfixes

Étant donnés un alphabet $\mathcal{S}$ et un arbre t de $\mathcal{T}_{\mathcal{S}}$, un mot de $\mathcal{S}^*$ peut être représenté par la concaténation des séquences de bits représentant chacune de ses lettres dans t , de la gauche vers la droite. Supposons que l'arbre t est l'exemple de la figure 1. Le mot « ADBDCD » est alors représenté par la séquence de bits 100|0|11|0|101|0. Remarque : les barres verticales ne servent qu'à rendre l'exemple plus lisible ; elles ne font pas partie de la séquence de bits et ne sont pas nécessaires pour retrouver le mot initial.

Question III.1. Soit t un arbre de $\mathcal{T}_{\mathcal{S}}$. Étant donnée une séquence de bits, montrez qu'il existe au plus un mot de $\mathcal{S}^*$ qui est représenté par cette séquence.

Question III.2. Supposons $\mathcal{S} \subseteq \mathbb{N}$. Définissez une fonction OCaml `decomp1 : int list -> tree -> int list` qui reçoit en argument une liste d'entiers 0 ou 1 et un arbre de $\mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste des éléments de $\mathcal{S}$ représentée par cette liste de bits. On sera attentif au cas où la liste en argument ne représente aucun mot de $\mathcal{S}^*$. Donnez et justifiez la complexité temporelle de cette fonction.

Cette fonction est dite de *décompression*. Supposons que $\mathcal{S}$ est $\{A, B, C, D\}$ et que la fonction q donne le nombre d'occurrences de chaque lettre dans le mot « ADBDCD », par exemple $q(D) = 3$. Dans ce cas, $c_q(t)$ vaut 11 pour l'arbre exemple de la Figure 1. Il s'agit de la longueur de la séquence 10001101010 représentant le mot « ADBDCD ». Plus généralement, $c_q(t)$ est la longueur de n'importe quelle séquence représentant un mot de $\mathcal{S}^q$ compressé avec l'arbre t . Il s'avère qu'il n'existe aucun arbre $t' \in \mathcal{T}_{\mathcal{S}}$ tel que $c_q(t') < 11$; t est donc optimal.

Étant donnée une fonction q , l'objectif de la partie suivante sera de construire un des arbres de $\mathcal{T}_{\mathcal{S}}$ offrant la meilleure *compression* des mots de $\mathcal{S}^q$, c'est-à-dire un arbre qui minimise c_q .

Partie IV. Arbres optimaux

Soit q une fonction dont le domaine inclut $\mathcal{S}$. Un arbre de $\mathcal{T}_{\mathcal{S}}$ est dit *optimal* pour $(q, \mathcal{S})$ s'il minimise c_q parmi tous les arbres de $\mathcal{T}_{\mathcal{S}}$. Autrement dit, un arbre optimal $t \in \mathcal{T}_{\mathcal{S}}$ vérifie $c_q(t) = \min_{t' \in \mathcal{T}_{\mathcal{S}}} c_q(t')$. De tels arbres optimaux existent et, dès lors que $|\mathcal{S}| \geq 2$, ils ne sont pas uniques.

Question IV.1. Soit $t = N(g, d) \in \mathcal{T}_{\mathcal{S}}$ un arbre optimal pour $(q, \mathcal{S})$. Soit $\mathcal{S}_g$ l'ensemble des lettres qui étiquettent les feuilles du sous-arbre g . Montrez que g est un arbre de $\mathcal{T}_{\mathcal{S}_g}$ optimal pour $(q, \mathcal{S}_g)$.

Malheureusement, cette propriété ne permet pas d'en déduire un algorithme de type « diviser pour régner » pour trouver l'arbre optimal. En effet, il faudrait que l'algorithme puisse efficacement deviner comment partitionner $\mathcal{S}$ entre les feuilles du sous-arbre gauche et celles du sous-arbre droit.

Question IV.2. Supposons qu'il existe une lettre $\sigma_0 \in \mathcal{S}$ telle que $q(\sigma_0) > \sum_{\sigma \in \mathcal{S} \setminus \{\sigma_0\}} q(\sigma)$. Montrez que, pour tout arbre de $\mathcal{T}_{\mathcal{S}}$ optimal pour $(q, \mathcal{S})$, la feuille $F(\sigma_0)$ est à profondeur 1, c'est-à-dire directement attachée à la racine.

Question IV.3. Soient σ_1 et σ_2 deux éléments différents de $\mathcal{S}$ tels que, pour tout $\sigma \in \mathcal{S} \setminus \{\sigma_1, \sigma_2\}$, on a $q(\sigma) \geq q(\sigma_1)$ et $q(\sigma) \geq q(\sigma_2)$. Soit $\mathcal{S}' = \mathcal{S} \setminus \{\sigma_1, \sigma_2\} \cup \{\sigma_3\}$ avec σ_3 un tout nouvel élément. On définit q' de telle sorte que $q'(\sigma_3) = q(\sigma_1) + q(\sigma_2)$ et $\forall \sigma \in \mathcal{S} \setminus \{\sigma_1, \sigma_2\}$, $q'(\sigma) = q(\sigma)$.

1. Montrez qu'il existe un arbre t de $\mathcal{T}_{\mathcal{S}}$ optimal pour $(q, \mathcal{S})$ ayant un nœud $N(F(\sigma_1), F(\sigma_2))$.
2. Soit un arbre t' de $\mathcal{T}_{\mathcal{S}'}$ optimal pour $(q', \mathcal{S}')$. Montrez que l'arbre $t \in \mathcal{T}_{\mathcal{S}}$ obtenu en remplaçant dans t' la feuille $F(\sigma_3)$ par $N(F(\sigma_1), F(\sigma_2))$ est optimal pour $(q, \mathcal{S})$.

On définit la fonction $\bar{q}$ en étendant la fonction q aux arbres de la façon suivante. Pour une feuille, $\bar{q}(F(\sigma))$ vaut $q(\sigma)$. Pour un nœud interne, $\bar{q}(N(g, d))$ vaut récursivement $\bar{q}(g) + \bar{q}(d)$. Remarque : en général, $c_q(t) \neq \bar{q}(t)$.

La question précédente montre qu'il est possible de construire un arbre optimal pour $(q, \mathcal{S})$ à l'aide de l'algorithme suivant. On initialise un ensemble d'arbres E avec toutes les feuilles $F(\sigma)$ avec $\sigma \in \mathcal{S}$. À chaque étape, on retire de E deux arbres t_1 et t_2 qui minimisent $\bar{q}$; puis on ajoute à E l'arbre $N(t_1, t_2)$. On répète cette procédure jusqu'à ce qu'il ne reste qu'un seul arbre dans E . Il s'agit alors d'un arbre optimal pour $(q, \mathcal{S})$.

Question IV.4. Implantez l'algorithme décrit ci-dessus en définissant les deux fonctions OCaml suivantes : `insert` et `optimal`.

1. La fonction `insert : (int * tree) -> (int * tree) list -> (int * tree) list` insère dans une liste son premier argument. La liste en argument est supposée triée par rapport à la première composante de chacune de ses paires. La liste renvoyée doit l'être aussi.
2. La fonction `optimal : (int * int) list -> tree` renvoie un arbre optimal. La liste passée en argument est de taille $|\mathcal{S}|$; elle contient toutes les paires $(\sigma, q(\sigma))$ pour $\sigma \in \mathcal{S}$; ces paires sont triées par valeur croissante de $q(\sigma)$.
3. Donnez et justifiez la complexité temporelle de la fonction `optimal`.

Les deux questions suivantes montrent que, quand la liste initiale des $(\sigma, q(\sigma))$ est triée par valeur croissante de $q(\sigma)$, de simples listes et/ou tableaux suffisent pour écrire une fonction `optimal` dont la complexité temporelle est en $O(|\mathcal{S}|)$.

Question IV.5. Montrez que, lors de l'exécution de l'algorithme décrit ci-dessus, les arbres $t = N(t_1, t_2)$ sont ajoutés à l'ensemble E par valeur croissante de $\bar{q}(t)$.

Question IV.6. Expliquez comment écrire la fonction `optimal` pour que sa complexité temporelle soit linéaire. Le code OCaml n'est pas demandé.

Cette partie a permis de calculer un arbre qui minimise la longueur c_q de la séquence de bits représentant un mot de $\mathcal{S}^q$. Mais pour décompresser ce mot, il faut disposer de l'arbre et donc l'avoir stocké quelque part. Parmi tous les arbres optimaux, il est donc important d'en choisir un qui nécessitera le moins de bits pour être représenté ; c'est l'objet de la partie suivante.

Partie V. Arbres canoniques

On suppose maintenant qu'il existe un ordre alphabétique noté $<$ sur les éléments de $\mathcal{S}$. Pour $\mathcal{S} \subseteq \mathbb{N}$, il s'agira de l'ordre usuel sur $\mathbb{N}$. Soit t un arbre de $\mathcal{T}_{\mathcal{S}}$. On définit la relation $\prec_t$ entre éléments de $\mathcal{S}$ de la façon suivante : pour toute paire $(\sigma_1, \sigma_2) \in \mathcal{S}^2$,

$$\sigma_1 \prec_t \sigma_2 \iff \ell_t(\sigma_1) < \ell_t(\sigma_2) \text{ ou } (\ell_t(\sigma_1) = \ell_t(\sigma_2) \text{ et } \sigma_1 < \sigma_2).$$

L'arbre t est dit *canonique* si, en parcourant les feuilles de la gauche vers la droite, leurs étiquettes respectent la relation $\prec_t$. Autrement dit, plus une feuille est à gauche, plus elle est proche de la racine ; et les étiquettes des feuilles à une même profondeur sont triées de gauche à droite par ordre alphabétique.

L'arbre de la figure 1 vérifie bien la deuxième partie de la propriété : les feuilles à une même profondeur sont triées de gauche à droite par ordre alphabétique. Mais il ne respecte pas la première partie : la feuille étiquetée par A est plus profonde que celle étiquetée par B alors qu'elle est plus à gauche. Cet arbre n'est donc pas canonique. L'arbre suivant est par contre canonique :

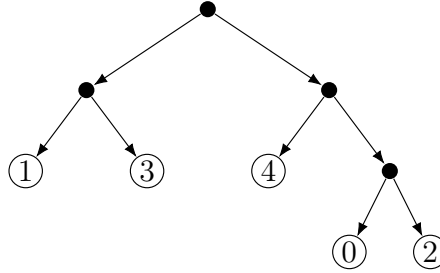


FIGURE 3 – Exemple d'arbre canonique.

Un arbre canonique peut être représenté par deux tableaux d'entiers. Le premier tableau associe à chaque indice i le nombre de lettres $\sigma \in \mathcal{S}$ telles que $\ell_t(\sigma) = i$. Le deuxième tableau contient les éléments de $\mathcal{S}$ obtenus en parcourant les feuilles de l'arbre de gauche à droite. Ainsi, l'arbre de $\mathcal{T}_{[0,4]}$ de la figure 3 est représenté par les deux tableaux $\llbracket 0; 0; 3; 2 \rrbracket$ et $\llbracket 1; 3; 4; 0; 2 \rrbracket$.

Question V.1. Définissez une fonction OCaml `canonical : int array -> int array -> tree` qui, étant donnés deux tels tableaux, renvoie l'arbre canonique qu'ils représentent, s'il existe. Donnez et justifiez la complexité temporelle de cette fonction.

Partie VI. Arbres alphabétiques

L'arbre de la figure 1 était optimal pour le mot « ADBDCD ». Un autre arbre optimal pour ce mot est le suivant :

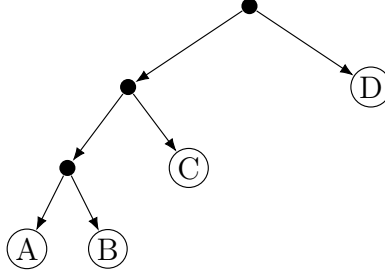


FIGURE 4 – Arbre à la fois optimal pour le mot « ADBDCD » et alphabétique.

Il n'est pas canonique mais il a une propriété très intéressante : ses feuilles sont dans l'ordre alphabétique. Cela signifie que lors du stockage de l'arbre, il n'est pas nécessaire de stocker les étiquettes des feuilles, il suffit de stocker la forme de l'arbre. Un tel arbre est dit *alphabétique*.

Plus généralement, un arbre de $\mathcal{T}_S$ est *alphabétique* si, en parcourant ses feuilles de gauche à droite, les étiquettes apparaissent dans l'ordre alphabétique. On note $\mathcal{A}_S$ l'ensemble de ces arbres. On dit d'un arbre qu'il est *alphabétique-optimal* pour (q, S) s'il minimise c_q parmi tous les arbres de $\mathcal{A}_S$. Remarque : un arbre alphabétique-optimal pour (q, S) n'est pas nécessairement optimal pour (q, S) .

Soit $n \in \mathbb{N}$ et $S = [0, n-1]$. On se donne une fonction $q : S \rightarrow \mathbb{N}$. Étant donnés deux entiers i et j tels que $0 \leq i \leq j < n$, on note $m_{i,j}$ la valeur de $c_q(t)$ pour n'importe quel arbre t de $\mathcal{A}_{[i,j]}$ alphabétique-optimal pour $(q, [i, j])$. En particulier, si t est un arbre de $\mathcal{A}_S$ alphabétique-optimal pour (q, S) , il vérifie $c_q(t) = m_{0,n-1}$.

Question VI.1. Exprimez la valeur de $m_{i,j}$ en fonction des valeurs de $m_{i',j'}$ avec $[i', j'] \subsetneq [i, j]$. Déduisez en une fonction OCaml `alpha_optimal : int array -> tree` qui calcule un arbre de $\mathcal{A}_{[0,n-1]}$ alphabétique-optimal pour $(q, [0, n-1])$. La fonction q est donnée par le tableau de taille n passé en argument. La complexité temporelle de la fonction devra être en $O(n^3)$.

Un arbre alphabétique de $\mathcal{A}_{[0,n-1]}$ peut être représenté par la séquence de $2n-1$ bits obtenue en effectuant un parcours en profondeur préfixe, c'est-à-dire que la racine d'un sous-arbre est visité avant ses feuilles et qu'un sous-arbre gauche est parcouru avant un sous-arbre droit. Pour chaque nœud, les chiffres 0 et 1 indiquent respectivement s'il s'agit d'un nœud interne ou d'une feuille. Par exemple, l'arbre de la figure 4 est représenté par la séquence 0001111.

Question VI.2. Définissez une fonction OCaml `alpha : int array -> tree` qui reçoit en argument un tableau de 0 et 1 et renvoie l'arbre alphabétique correspondant. On sera attentif au cas où le tableau en argument ne représente aucun arbre. Cette fonction devra avoir une complexité temporelle en $O(n)$ avec n la taille du tableau.

Partie VII. Codes arithmétiques

Soient un alphabet $\mathcal{S}$ et une fonction $q : \mathcal{S} \rightarrow \mathbb{N}$. Comme précédemment, $\mathcal{S}^q$ est le sous-ensemble des mots de $\mathcal{S}^*$ qui contiennent exactement $q(\sigma)$ occurrences de σ pour chaque $\sigma \in \mathcal{S}$.

Question VII.1. Supposons que $\mathcal{S}$ est $\{A, B, C\}$ et que q satisfait $q(A) = 15$, $q(B) = 4$, $q(C) = 1$.

1. Combien de bits faut-il pour représenter un mot de $\mathcal{S}^q$ en utilisant un arbre optimal pour $(q, \mathcal{S})$?
2. Combien y a-t-il de mots dans $\mathcal{S}^q$? On pourra exprimer ce nombre sous forme d'un produit de nombres premiers.
3. Montrez que 17 bits suffisent pour représenter chaque entier entre 0 et $|\mathcal{S}^q| - 1$ (et donc n'importe quel mot de $\mathcal{S}^q$). Remarque : $15 \cdot 17 = 2^8 - 1$.

L'exemple de la question précédente montre que, dans certains cas, par exemple quand une lettre est bien plus fréquente que les autres, l'utilisation d'un code préfixe n'est pas l'approche optimale. Il faut alors se tourner vers d'autres mécanismes de compression.

On se restreint maintenant au cas où $\mathcal{S} = [0, n - 1]$. Soit $N = \sum_{\sigma \in \mathcal{S}} q(\sigma)$. La fonction $E_q : \mathbb{N} \times \mathcal{S} \rightarrow \mathbb{N}$ est définie de la façon suivante pour $q(\sigma) \neq 0$:

$$E_q(x, \sigma) = \lfloor x/q(\sigma) \rfloor \cdot N + (x \bmod q(\sigma)) + \sum_{0 \leq k < \sigma} q(k),$$

où $\lfloor a \rfloor$ désigne la partie entière de a .

La fonction E_q peut être étendue en une fonction $C_q : \mathbb{N} \times \mathcal{S}^* \rightarrow \mathbb{N}$ qui travaille sur les mots de la façon suivante :

$$C_q(x, \sigma_0 \sigma_1 \dots \sigma_k) = E_q(\dots E_q(E_q(x, \sigma_0), \sigma_1), \dots \sigma_k).$$

Ainsi, après avoir choisi x arbitrairement, un mot $\sigma_0 \dots \sigma_{N-1} \in \mathcal{S}^q$ peut être compressé en un entier $y = C_q(x, \sigma_0 \dots \sigma_{N-1})$. Il est alors possible de retrouver le mot original à partir de y en calculant progressivement σ_{N-1} , σ_{N-2} , etc, jusqu'à σ_0 .

Considérons par exemple le mot « 0120000 » ($N = 7$, $q = \llbracket 5, 1, 1 \rrbracket$). Si l'on part de $x = 0$, sa version compressée sera l'entier 151 :

$$0 \xrightarrow{0} 0 \xrightarrow{1} 5 \xrightarrow{2} 41 \xrightarrow{0} 57 \xrightarrow{0} 79 \xrightarrow{0} 109 \xrightarrow{0} 151$$

où $x \xrightarrow{\sigma} y$ signifie $E_q(x, \sigma) = y$.

Question VII.2. Définissez une fonction OCaml `decomp2 : int array -> int -> int -> int * int array` qui reçoit en argument un tableau représentant q et deux entiers y et k , et renvoie un entier x et un tableau $[\sigma_0, \sigma_1, \dots, \sigma_{k-1}]$ tels que $E_q(x, \sigma_0 \sigma_1 \dots \sigma_{k-1}) = y$.

Le nombre de bits de l'entier $C_q(x, \sigma_0 \dots \sigma_{N-1})$ est proche de l'optimum théorique $\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \log_2(N/q(\sigma))$ mais un problème se pose en pratique. Sauf pour de tout petits mots sur de tout petits alphabets, le calcul de $C_q(x, \sigma_0 \dots \sigma_{N-1})$ va nécessiter de manipuler des entiers très grands. Pour éviter ce problème, une solution consiste, avant chaque appel à E_q , à se débarrasser d'un certain nombre k_i de bits de poids faible. Plus précisément, on construit les suites (x_n) et (y_n) suivantes :

$$\begin{cases} y_i &= \lfloor x_i / 2^{k_i} \rfloor \\ x_{i+1} &= E_q(y_i, \sigma_i) \end{cases}$$

Comme précédemment, la valeur de x_0 est fixée arbitrairement. Le mot $\sigma_0 \dots \sigma_{N-1}$ peut alors être représenté par d'une part x_N et d'autre part la concaténation des séquences de bits suivantes, les bits de poids forts apparaissant en premier :

$$\underbrace{x_{N-1} \bmod 2^{k_{N-1}}}_{k_{N-1} \text{ bits}}; \quad \dots; \quad \underbrace{x_2 \bmod 2^{k_2}}_{k_2 \text{ bits}}; \quad \underbrace{x_1 \bmod 2^{k_1}}_{k_1 \text{ bits}}.$$

Pour chaque i en ordre décroissant, le décompresseur déduit de x_{i+1} les valeurs de y_i et σ_i (par exemple avec `decomp2`, question **VII.2**, avec $k = 1$), puis il lit k_i bits dans le mot compressé et les concatène à y_i pour obtenir x_i , et ainsi de suite jusqu'à avoir décompressé tout le mot.

Dans la suite, on supposera que N est une puissance de deux : $N = 2^K$. On supposera par ailleurs que le processeur n'est efficace que pour des entiers ne dépassant pas 2^B pour un certain B bien plus grand que K . En particulier, $x_{i+1} = E_q(y_i, \sigma_i)$ ne doit jamais atteindre cette borne 2^B lors de la compression.

Pour que le taux de compression se rapproche de l'optimum théorique, il faut que chaque k_i soit le plus petit possible (idéalement 0) et donc que chaque y_i soit le plus grand possible. Par ailleurs, les différents k_i ne font pas partie du mot compressé. Autrement dit, en ne connaissant que y_i , le décompresseur doit pouvoir deviner le nombre k_i de bits qui doivent être lus dans le mot compressé pour reconstruire x_i . Une solution correcte mais mauvaise serait, par exemple, de fixer tous les k_i à la constante K ; le décompresseur pourrait alors deviner trivialement les k_i mais le mot résultant ne serait absolument pas compressé.

Question VII.3. Proposez une façon pour le compresseur de choisir k_i en fonction de x_i et σ_i . Expliquez comment le décompresseur choisit k_i en fonction de y_i et prouvez que ce choix correspond bien à celui du compresseur. Si cela a une importance, expliquez comment le compresseur doit choisir x_0 .

Remarque : la contrainte imposant que N soit une puissance de deux ne pose aucune difficulté en pratique. En effet, en matière de compression entropique, la seule chose qui importe est que $q(\sigma)/N$ soit proche de la proportion de la lettre σ dans le mot à compresser.

Fin du sujet.

ECOLES NORMALES SUPERIEURES

CONCOURS D'ADMISSION 2023

**VENDREDI 21 AVRIL 2023
14h00 - 18h00**

FILIERES MP et MPI

Epreuve n° 10

INFO-FONDAMENTALE (ULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas
autorisée pour cette épreuve***

Épreuve d'informatique fondamentale

Concision et ambiguïté

Le sujet porte sur les automates finis, dont la définition est rappelée dans le préambule. Le sujet s'intéresse à la propriété d'ambiguïté des automates finis, propriété également définie dans le préambule. La première partie lie les notions de déterminisme et d'ambiguïté d'un automate fini en utilisant la notion de miroir d'un automate. La deuxième partie amène à définir un algorithme qui teste si un automate est ambigu et vous demande de déterminer sa complexité asymptotique. La troisième et la quatrième partie étudient la concision des automates non ambigus et ambigus.

Les parties 1 et 2 sont indépendantes ; il est conseillé de les traiter en premier car les parties 3 et 4 en dépendent. Il est permis d'admettre les réponses à certaines questions pour répondre aux suivantes.

Préambule

On note Σ un *alphabet* fini, c'est à dire un ensemble fini de symboles appelés *lettres*. Un *mot* sur Σ est une suite finie de lettres. On note Σ^n l'ensemble des mots de longueur n sur Σ et Σ^* l'ensemble de tous les mots sur Σ . Soient u, v deux mots sur Σ , on note $u \cdot v$ la *concaténation* de u et v . Un *langage* sur Σ est un sous-ensemble de Σ^* .

Un automate sur Σ est un tuple $\mathcal{A} = (Q, T, I, F)$ où :

- Q est un ensemble fini de symboles appelés *états* ;
- $T \subseteq Q \times \Sigma \times Q$ est appelé ensemble des *transitions* ;
- $I \subseteq Q$ est l'ensemble des états *initiaux* ;
- $F \subseteq Q$ est l'ensemble des états *finaux*.

La Figure 1 représente graphiquement trois automates. Les symboles dans Q sont encerclés, avec deux cercles pour les symboles dans F . Une transition $(q, a, q') \in T$ est représentée par une flèche étiquetée par $a \in \Sigma$, allant de l'état source q à l'état destination q' . Les états initiaux sont indiqués par une flèche sans état source.

Un calcul de $\mathcal{A}$ sur un mot $w = a_0 \dots a_{n-1} \in \Sigma^*$ est une suite finie d'états $q_0 \dots q_n \in Q^*$ telle que $q_0 \in I$ et pour tout $i < n$, $(q_i, a_i, q_{i+1}) \in T$. Un tel calcul est dit *acceptant* si $q_n \in F$. On dit alors que $\mathcal{A}$ accepte w . Le langage de $\mathcal{A}$, noté $L(\mathcal{A})$, est l'ensemble des mots acceptés par $\mathcal{A}$.

Un automate $\mathcal{A}$ est dit *déterministe* si $|I| \leq 1$ et pour tout $(q, a) \in Q \times \Sigma$, $|\{q' \mid (q, a, q') \in T\}| \leq 1$.

Un automate $\mathcal{A}$ est dit *complet* si $|I| \geq 1$ et pour tout $(q, a) \in Q \times \Sigma$, $|\{q' \mid (q, a, q') \in T\}| \geq 1$.

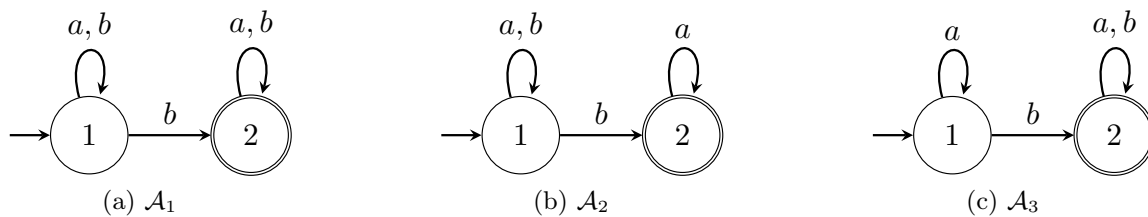


FIGURE 1 – Trois automates reconnaissant le même langage

Soient $w \in \Sigma^*$ et $\mathcal{A}$ un automate. Le mot w est dit *ambigu* pour $\mathcal{A}$ s'il existe deux calculs acceptants ρ et ρ' de $\mathcal{A}$ sur w avec $\rho \neq \rho'$. On définit $d_{\mathcal{A}}(w)$, le *degré d'ambiguïté* de w dans $\mathcal{A}$, comme étant le nombre de calculs acceptants différents de $\mathcal{A}$ sur w . Ainsi, w est ambigu pour $\mathcal{A}$ si et seulement si $d_{\mathcal{A}}(w) > 1$. On note $\text{Amb}(\mathcal{A})$ l'ensemble des mots ambigus pour $\mathcal{A}$. L'automate $\mathcal{A}$ est dit *ambigu* si $\text{Amb}(\mathcal{A}) \neq \emptyset$.

Ce sujet s'intéresse tout particulièrement aux automates *non ambigus*.

1 Déterminisme et ambiguïté

Question 1.1 Les trois automates de la Figure 1 acceptent le même langage. Donnez-en, sans justification, une description intuitive.

Question 1.2 Pour chacun des automates de la Figure 1, dites s'il est déterministe ou non déterministe. Justifiez vos affirmations.

Question 1.3 Pour l'automate $\mathcal{A}_1$ de la Figure 1, calculez $\text{Amb}(\mathcal{A}_1)$ et déduisez en si $\mathcal{A}_1$ est ambigu ou non. Faites de même pour les automates $\mathcal{A}_2$ et $\mathcal{A}_3$.

Soit $\mathcal{A}$ un automate. On note $\tilde{\mathcal{A}} = (\tilde{Q}, \tilde{T}, \tilde{I}, \tilde{F})$ l'*automate miroir* de $\mathcal{A}$, défini par :

- $\tilde{Q} = Q$;
- $\tilde{I} = F$;
- $\tilde{F} = I$;
- $\tilde{T} = \{(t, a, s) \mid (s, a, t) \in T\}$.

Un automate est dit *co-déterministe* si son automate miroir est déterministe.

Question 1.4 Soit $\mathcal{A}$ un automate, $w \in L(\mathcal{A})$ un mot accepté par $\mathcal{A}$ et $q_0 \dots q_n$ un calcul acceptant de $\mathcal{A}$ sur w . Montrez qu'il existe un mot $\tilde{w}$ tel que $q_n \dots q_0$ soit un calcul acceptant de $\tilde{\mathcal{A}}$ sur $\tilde{w}$.

Question 1.5 Montrez qu'un automate $\mathcal{A}$ est ambigu si et seulement si $\tilde{\mathcal{A}}$ est ambigu.

Question 1.6 Montrez que si un automate $\mathcal{A}$ est déterministe, alors $\mathcal{A}$ n'est pas ambigu.

Question 1.7 Montrez que si un automate $\mathcal{A}$ est co-déterministe, alors $\mathcal{A}$ n'est pas ambigu.

Question 1.8 Pour chacune des questions suivantes, donnez un automate $\mathcal{A}$ ayant au plus 4 états et respectant les propriétés demandées. Justifiez vos réponses.

- (i) $\mathcal{A}$ est non ambigu mais ni déterministe, ni co-déterministe ;
- (ii) $L(\mathcal{A})$ est infini et $\text{Amb}(\mathcal{A}) = L(\mathcal{A})$;
- (iii) $\text{Amb}(\mathcal{A})$ est infini, et $\text{Amb}(\mathcal{A}) \neq L(\mathcal{A})$.

2 Test d'ambiguïté

Le but de cette partie est d'obtenir un algorithme qui teste si un automate est ambigu et de déterminer sa complexité asymptotique.

Dans cette partie, $\mathcal{A}$ est un automate (Q, T, I, F) tel que $L(\mathcal{A}) \neq \emptyset$.

2.1 Une construction utile

En utilisant $\mathcal{A}$, on définit l'automate $\hat{\mathcal{A}} = (\hat{Q}, \hat{T}, \hat{I}, \hat{F})$ comme suit :

- $\hat{Q} = Q \times Q \times \{0, 1\}$;
- $\hat{I} = \{(i, i, 0) \mid i \in I\} \cup \{(i, i', 1) \mid i \in I, i' \in I, i \neq i'\}$;
- $\hat{F} = \{(f, f', 1) \mid f \in F, f' \in F\}$;
- $\hat{T} = T_1 \cup T_2 \cup T_3$ avec :
 - $T_1 = \{((s, s, 0), a, (t, t, 0)) \mid (s, a, t) \in T\}$;
 - $T_2 = \{((s, s, 0), a, (t, t', 1)) \mid (s, a, t) \in T, (s, a, t') \in T, t \neq t'\}$;
 - $T_3 = \{((s, s', 1), a, (t, t', 1)) \mid (s, a, t) \in T, (s', a, t') \in T\}$.

Question 2.1 Soit $\mathcal{A}_1$ le premier automate de la figure Figure 1. Construisez l'automate $\hat{\mathcal{A}}_1$. Il est inutile de faire figurer les états qui ne sont pas accessibles à partir d'un état initial. Donnez, sans justification, le langage $L(\hat{\mathcal{A}}_1)$.

Question 2.2 Soient $w \in \Sigma^*$ un mot et $\rho = (q_0, q'_0, b_0) \dots (q_n, q'_n, b_n)$ un calcul de $\hat{\mathcal{A}}$ sur w . On pose $\mu = q_0 \dots q_n$ et $\mu' = q'_0 \dots q'_n$.

- (i) Montrez que μ et μ' sont des calculs de $\mathcal{A}$ sur w .
- (ii) Montrez que $b_n = 0$ si et seulement si $\mu = \mu'$.
- (iii) Montrez que, si ρ est acceptant, alors μ et μ' sont acceptants.
- (iv) Montrez qu'il existe un calcul ρ tel que μ et μ' sont acceptants mais ρ ne l'est pas.

Question 2.3 Montrez que $L(\hat{\mathcal{A}}) = \text{Amb}(\mathcal{A})$.

2.2 L'algorithme

Pour deux fonctions $f, g : \mathbb{N} \rightarrow \mathbb{N}$, on dit que g est une borne asymptotique de f , et on le note par $f \in \mathcal{O}(g)$, s'il existe deux constantes strictement positives $n_0 \in \mathbb{N}$ et $c \in \mathbb{N}$ telles que pour tout $n \geq n_0$, $f(n) \leq c \times g(n)$. Cette définition se généralise naturellement à des fonctions f et g avec plusieurs paramètres.

Question 2.4 Pour chaque ensemble $\hat{Q}$, $\hat{T}$, $\hat{I}$ et $\hat{F}$, donnez une borne asymptotique au nombre d'éléments qu'il contient en fonction des tailles de Q , T , I et F .

Question 2.5 Donnez un algorithme qui a comme entrée $\mathcal{A}$ et comme sortie $\hat{\mathcal{A}}$ en précisant :

- (i) quelle structure de données classique (matrice d'adjacence ou liste d'adjacence) est utilisée pour représenter $\mathcal{A}$ et $\hat{\mathcal{A}}$,
- (ii) une borne asymptotique de la complexité en temps d'exécution de cet algorithme en fonction de la somme des tailles des ensembles Q , T , I et F .

Question 2.6 Décrivez une méthode permettant de tester si $\mathcal{A}$ est ambigu en utilisant l'automate $\hat{\mathcal{A}}$. Donnez une borne asymptotique de la complexité en temps de cette méthode en fonction de la somme des tailles des ensembles Q , T , I et F .

2.3 Généralisation

Soit k un entier strictement positif.

Question 2.7 Soit $w \in L(\mathcal{A})$ un mot de longueur k .

- (i) Donnez, en fonction de k et $|Q|$, une borne supérieure sur le degré d'ambiguïté de w dans $\mathcal{A}$.
- (ii) Donnez un automate $\mathcal{A}$ et un mot de longueur k pour lesquels la borne supérieure indiquée ci-dessus est atteinte.

Question 2.8 On pose $\text{Amb}_{\geq k}(\mathcal{A}) = \{w \in L(\mathcal{A}) \mid d_{\mathcal{A}}(w) \geq k\}$. Montrez que $\text{Amb}_{\geq k}(\mathcal{A})$ est régulier.

Question 2.9 On pose $\text{Amb}_k(\mathcal{A}) = \{w \in L(\mathcal{A}) \mid d_{\mathcal{A}}(w) = k\}$. Montrez que $\text{Amb}_k(\mathcal{A})$ est régulier.

3 Concision des automates non ambigus

Le but de cette partie est de prouver que les automates non ambigus peuvent être exponentiellement plus concis que leurs équivalents déterministes et complets.

Dans toute cette partie, on fixe l'alphabet $\Sigma = \{a, b\}$. Pour $n \geq 1$ un entier, on pose $L_n = \{w_1 \cdot b \cdot w_2 \mid w_1, w_2 \in \Sigma^*, |w_2| = n - 1\}$, le langage des mots dont la n -ième lettre en partant de la fin est un b .

Question 3.1 *Donnez un automate non ambigu acceptant L_3 , puis donnez un automate déterministe et complet acceptant L_3 .*

Question 3.2 *Montrez que pour tout $n \geq 1$ entier, il existe un automate non ambigu $\mathcal{A}$ avec $n + 1$ états tel que $L(\mathcal{A}) = L_n$.*

Question 3.3 *Montrez que pour tout $n \geq 1$ entier, il existe un automate déterministe et complet $\mathcal{B}$ avec 2^n états tel que $L(\mathcal{B}) = L_n$.*

On veut à présent prouver que tout automate déterministe et complet acceptant L_n a au moins 2^n états.

Question 3.4 *Soit $\mathcal{B}$ un automate déterministe et complet. Soit $w \in \Sigma^*$. Montrez qu'il existe un unique calcul, non nécessairement acceptant, de $\mathcal{B}$ sur w .*

Soit $\mathcal{B}$ un automate déterministe et complet et $w \in \Sigma^*$. On note q_w l'état atteint par l'unique calcul de $\mathcal{B}$ sur w , c'est-à-dire l'état q_m tel que $q_0 \dots q_m$ est un calcul de $\mathcal{B}$ sur w .

Question 3.5 *Soit $n \in \mathbb{N}$ et $\mathcal{B}$ un automate déterministe et complet reconnaissant L_n . Soient w et w' deux mots de Σ^* de longueur n . Montrez que $q_w = q_{w'}$ si et seulement si $w = w'$.*

Question 3.6 *Soit $n \in \mathbb{N}$. Montrez que tout automate déterministe et complet reconnaissant L_n a au moins 2^n états.*

4 Concision des automates ambigus

Le but de cette partie est de prouver que les automates ambigus peuvent être exponentiellement plus concis que leurs équivalents non ambigus.

Pour $n \in \mathbb{N}$, on pose Σ_n un alphabet à n lettres et K_n le langage des mots sur Σ_n^* dont au moins une lettre apparaît au moins deux fois, c'est-à-dire :

$$K_n = \{w_1 \cdot x \cdot w_2 \cdot x \cdot w_3 \mid w_1, w_2, w_3 \in \Sigma_n^*, x \in \Sigma_n\}$$

Question 4.1 Pour $\Sigma_3 = \{a, b, c\}$,

- (i) donnez un automate acceptant K_3 et ayant au plus 5 états,
- (ii) donnez un automate non ambigu acceptant K_3 et ayant au plus 9 états.

Question 4.2 Montrez que pour tout $n \in \mathbb{N}$, il existe un automate avec au plus $n + 2$ états acceptant K_n .

Question 4.3 Montrez que pour tout $n \in \mathbb{N}$, il existe un automate non ambigu avec au plus $2^n + 1$ états acceptant K_n .

On veut à présent prouver que tout automate non ambigu acceptant K_n a au moins $2^n + 1$ états. Jusqu'à la fin de cette partie, on fixe un entier $n \geq 2$ et un automate non ambigu $\mathcal{A} = (Q, T, I, F)$ acceptant K_n .

Soient u et v deux mots de Σ_n^* . Lorsque $u \cdot v \in K_n$, on note $q_{u,v}$ l'état de $\mathcal{A}$ atteint après avoir lu u lors de l'unique calcul acceptant de $\mathcal{A}$ sur $u \cdot v$. Autrement dit, soit $\ell = |u|$ et $q_0 \dots q_m$ l'unique calcul acceptant de $\mathcal{A}$ sur $u \cdot v$, alors $q_{u,v} = q_\ell$.

Question 4.4 Soient u, u' et v, v' quatre mots de Σ_n^* tels que $u \cdot v \in K_n$ et $u' \cdot v' \in K_n$. On suppose que $q_{u,v} = q_{u',v'}$.

- (i) Montrez que $u \cdot v' \in K_n$ et $u' \cdot v \in K_n$.
- (ii) Montrez que $q_{u,v} = q_{u,v'} = q_{u',v} = q_{u',v'}$.

L'objectif à présent est de prouver que ces contraintes garantissent que $\mathcal{A}$ a au moins 2^n états.

Soit $s = (s_i)$ une suite de mots et α une lettre. On note $s \cdot \alpha$ la suite $(s_i \cdot \alpha)$, c'est-à-dire la suite obtenue en ajoutant α à la fin de chaque mot de s .

On fixe un ordre total $<$ sur Σ_n , et on note $\Sigma_n = \{\alpha_1, \dots, \alpha_n\}$ avec pour tout $1 \leq i < j \leq n$, $\alpha_i < \alpha_j$. On construit une famille $s^0, \dots, s^n$ de suites de mots de Σ_n comme suit :

- s^0 est la suite contenant uniquement ε ;

— Pour $0 < i \leq n$, $s^i = s^{i-1} \cdot s^{i-1} \cdot \alpha_i$, la suite constituée de s^{i-1} , suivie de la suite $s^{i-1} \cdot \alpha_i$.

Finalement, on pose $s = s^n$. Par exemple, pour $n = 2$, $\Sigma_2 = \{a, b\}$ et $a < b$, la suite s contient 4 éléments : $s_0 = \varepsilon$, $s_1 = a$, $s_2 = b$, $s_3 = ab$.

Finalement, on remarque que s contient 2^n éléments et on pose M_n la matrice de dimension $2^n \times 2^n$ à coefficients en $\mathbb{Z}$ définie par :

$$M_n[i, j] = \begin{cases} 1 & \text{si } s_i \text{ et } s_j \text{ ont une lettre en commun} \\ 0 & \text{sinon} \end{cases}$$

Par exemple, M_2 est donnée ci-dessous :

$$M_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

Question 4.5 Montrez que M_n peut s'écrire sous la forme :

$$M_n = \begin{pmatrix} M_{n-1} & M_{n-1} \\ M_{n-1} & \mathbf{1}_{n-1} \end{pmatrix}$$

où, pour $n \geq 1$, $\mathbf{1}_n$ est la matrice carrée de dimension $2^n \times 2^n$ dont tous les coefficients valent 1.

Question 4.6 Soient $i < 2^n$ et $j < 2^n$. Montrez que $M_n[i, j] = 1$ si et seulement si $s_i \cdot s_j \in K_n$.

À chaque état q de $\mathcal{A}$, on associe le vecteur colonne v_q de dimension 2^n défini par :

$$v_q[i] = \begin{cases} 1 & \text{s'il existe } j < 2^n \text{ tel que } q = q_{s_i, s_j} \\ 0 & \text{sinon} \end{cases}$$

Question 4.7 Montrez que l'ensemble de vecteurs $(v_q)_{q \in Q}$ est une famille génératrice du sous-espace vectoriel de $\mathbb{Z}^n$ engendré par les vecteurs colonnes de M_n .

Indication : Soit u_j la j -ème colonne de M_n . On pourra montrer que u_j est une combinaison linéaire de vecteurs de $(v_q)_{q \in Q}$ en prouvant que :

$$u_j = \sum_{\substack{q \in Q \\ \exists i, q = q_{s_i, s_j}}} v_q$$

Question 4.8 *Montrez que M_n est de rang $2^n - 1$.*

Question 4.9 *En déduire que $\mathcal{A}$ contient au moins $2^n - 1$ états q tels que $v_q \neq 0$.*

Question 4.10 *Montrez que si $v_q \neq 0$, alors $q \notin I$ et $q \notin F$.*

Question 4.11 *Conclure que $\mathcal{A}$ a au moins $2^n + 1$ états.*

A2023 – INFO MPI I

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

Concours Mines-Télécom

CONCOURS 2023

PREMIÈRE ÉPREUVE D'INFORMATIQUE

Durée de l'épreuve : 3 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE I - MPI

L'énoncé de cette épreuve comporte 9 pages de texte.

Cette épreuve concerne uniquement les candidats de la filière MPI.

*Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé,
il le signale sur sa copie et poursuit sa composition en expliquant les raisons
des initiatives qu'il est amené à prendre.*

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.
Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

Présentation du sujet

L'épreuve est composée d'un problème unique comportant 32 questions divisées en trois sections. L'objectif du problème est de construire des *listes à accès direct* : une liste à accès direct est un type de donnée abstrait qui permet, d'une part, d'empiler et de dépiler efficacement un élément en tête de liste et, d'autre part, d'accéder efficacement au k^{e} élément de la liste, pour n'importe quel indice k valide.

Dans la première section (page 1), nous étudions un système de numération : la représentation binaire gauche des entiers naturels. Dans la deuxième section (page 4), nous étudions les arbres binaires parfaits. Dans la troisième section (page 6), nous implémentons le type liste à accès direct par la structure de données concrète *liste gauche*, que nous construisons en exploitant les résultats obtenus dans les sections précédentes.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désignera la même entité, mais du point de vue mathématique avec la police en italique (par exemple n) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat. Des rappels de programmation sont faits en annexe et peuvent être utilisés directement.

Selon les questions, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Il est inutile de rappeler que les entêtes `<assert.h>`, `<stdbool.h>`, etc. doivent être inclus.

Quand l'énoncé demande de coder une fonction, sauf indication explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites. On suppose que le type `int` n'est jamais sujet à des débordements.

Le barème tient compte de la clarté des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle. Lorsqu'une réponse en pseudo-code est permise, seule la logique de programmation est évaluée, même dans le cas où un code en C a été fourni en guise de réponse.

1. Représentation binaire gauche des entiers naturels

1.1. Mise en place

Soient m un entier naturel et N un entier naturel non nul. Il est classique d'appeler *représentation binaire standard de l'entier m sur N chiffres*, ou plus simplement *représentation standard*, toute suite finie $b = (b_n)_{0 \leq n < N}$ de longueur N telle que, pour tout indice n compris entre 0 et $N - 1$, le chiffre b_n appartient à $\{0, 1\}$ et l'égalité suivante est vérifiée

$$m = \sum_{n=0}^{N-1} b_n 2^n.$$

Première épreuve d'informatique MPI 2023

Définition : Nous appelons *représentation binaire gauche de l'entier m sur N chiffres* toute suite finie $g = (g_n)_{0 \leq n < N}$ de longueur N telle que,

- (i) pour tout indice n compris entre 0 et $N - 1$, le chiffre g_n appartient à $\{0, 1, 2\}$,
- (ii) l'égalité suivante est satisfaite

$$m = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1),$$

- (iii) le chiffre « 2 » n'apparaît qu'au plus une fois parmi les chiffres $(g_n)_{0 \leq n < N}$,
- (iv) s'il existe une position p tel que le chiffre g_p est 2, alors, pour tout indice n compris entre 0 et $p - 1$, le chiffre g_n est nul.

De manière plus courte, nous parlons simplement de *représentation gauche*.

La figure 1 ci-dessous donne une représentation standard et une représentation gauche sur quatre chiffres des seize premiers entiers. Conformément à l'usage habituel, nous écrivons toute représentation, qu'elle soit standard ou gauche, sous la forme d'un mot $b_{N-1} \cdots b_0$ ou $g_{N-1} \cdots g_0$ dans lequel les chiffres de poids faibles sont écrits à droite.

Entier	Repr. standard	Repr. gauche	Entier	Repr. standard	Repr. gauche
0	0000	0000	8	1000	0101
1	0001	0001	9	1001	0102
2	0010	0002	10	1010	0110
3	0011	0010	11	1011	0111
4	0100	0011	12	1100	0112
5	0101	0012	13	1101	0120
6	0110	0020	14	1110	0200
7	0111	0100	15	1111	1000

FIGURE 1 – Représentations standard et gauche des seize premiers entiers

□ 1 – Soit c un entier. Donner la représentation standard de l'entier dont une représentation gauche est $10 \cdots 0$ (avec c chiffres nuls) en justifiant sommairement. Faire de même avec l'entier dont une représentation gauche est $20 \cdots 0$ (avec c chiffres nuls).

□ 2 – Déterminer, en justifiant, le plus grand entier naturel M_N qui admet une représentation gauche sur N chiffres. Préciser la représentation gauche de cet entier.

Définition : Soit n_0 un indice compris entre 0 et $N - 1$. On dit que l'indice n_0 est la *position du chiffre de plus fort poids* d'une représentation $g_{N-1} \cdots g_0$ si l'indice n_0 est le plus petit entier tel que, pour tout indice $n > n_0$, le chiffre g_n est nul. On appelle le chiffre g_{n_0} le *chiffre de plus fort poids*.

□ 3 – Soient N un entier naturel non nul, $g = g_{N-1} \cdots g_0$ et $h = h_{N-1} \cdots h_0$ deux représentations gauches d'un même entier m . Démontrer que les chiffres de plus fort poids de g et de h sont de même valeur et à la même position.

□ 4 – Démontrer que tout entier appartenant à l'intervalle $\llbracket 0, M_N \rrbracket$, où l'entier M_N a été introduit à la question 2, ne possède au plus qu'une seule représentation gauche sur N chiffres.

Première épreuve d'informatique MPI 2023

Indication C : L'entier N est déclaré comme constante globale. Nous utilisons la structure C déclarée comme suit pour écrire la représentation gauche sur N chiffres d'un entier $g_{N-1} \cdots g_0$:

```
1.  const int N = 8;
2.
3.  struct RepGauche {
4.      int position;
5.      bool chiffres[N];
6.  };
7.  typedef struct RepGauche rg;
```

Le champ `position` repère la position éventuelle du chiffre 2; il vaut -1 au cas où le chiffre 2 n'apparaît pas. Pour tout indice n compris entre 0 et $N - 1$, la n^e case du champ `chiffres` vaut `true` si le chiffre g_n est non-nul et vaut `false` sinon.

Par exemple, les entiers 15 et 21 ont pour représentation gauche les variables `entier_15` et `entier_21` suivantes :

```
8.  rg entier_15 = { .position = -1,
9.                  .chiffres = { 0, 0, 0, 1, 0, 0, 0, 0 } };
10. rg entier_21 = { .position = 1,
11.                 .chiffres = { 0, 1, 0, 1, 0, 0, 0, 0 } };
```

□ 5 – Formaliser sous la forme d'un ou de plusieurs invariants le fait qu'une valeur C de type **rg** est la représentation gauche d'un entier.

□ 6 – Écrire une fonction C **int** `rg_to_int(rg g)`, qui renvoie l'entier dont g est la représentation gauche. On supposera que l'invariant de la question 5 est satisfait.

1.2. Incrémentation et décrémentation

Nous proposons l'algorithme suivant :

ALGORITHME MYSTÈRE :

Entrée : Représentation gauche $g = g_{N-1} \cdots g_0$ d'un certain entier m .

Effet :

- Si aucun des chiffres $(g_n)_{0 \leq n < N}$ ne vaut 2, changer le chiffre g_0 en $g_0 + 1$.
- Sinon, en notant p la position du chiffre 2, changer le chiffre g_p en 0 et le chiffre g_{p+1} en $g_{p+1} + 1$.

Nous notons m' l'entier dont la représentation gauche est g après exécution de l'algorithme.

FIGURE 2 – Un algorithme

□ 7 – Vérifier que l'invariant de la question 5 n'est pas rompu par l'algorithme mystère (cf. figure 2). Avec les notations m et m' de la figure 2, caractériser, en fonction de l'entier m , la valeur de l'entier m' .

Première épreuve d'informatique MPI 2023

- 8 – Écrire une fonction C `bool rg_incr(rg *s)` dont la spécification suit :
- Précondition* : La variable s est un pointeur vers la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m + 1$.
- Valeur de retour* : Booléen `true` si l'incrément de m peut avoir lieu et `false` si un débordement se produit car $m + 1$ n'est pas représentable sur le même nombre de chiffres.
- 9 – Calculer la complexité en temps dans le pire des cas de la fonction `rg_incr` en fonction de N . Comparer avec la complexité de la même opération sur la représentation standard.
- 10 – Écrire une fonction C `bool rg_decr(rg *s)` dont la spécification suit :
- Précondition* : Le pointeur s désigne la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m - 1$.
- Valeur de retour* : Booléen `true` si la décrémentation de m peut avoir lieu et `false` si un débordement se produit car m est nul.
- Il est recommandé d'expliquer son intention avant de donner son code.

2. Arbres binaires parfaits

2.1. Opérations sur les arbres binaires

Indication C : Nous représentons des arbres binaires à valeurs entières au moyen du type C `arb` suivant, qui est un pointeur vers une structure contenant la valeur entière dans le champ `valeur` et les deux fils dans les champs `fils_g` et `fils_d`. L'arbre vide se représente par le pointeur `NULL`.

```

12.  typedef struct Noeud *arb;
13.  struct Noeud {
14.      int valeur;
15.      arb fils_g;
16.      arb fils_d;
17.  };

```

L'arbre vide est, par convention, de hauteur -1 .

- 11 – Écrire une fonction C `int hauteur(arb a)` qui calcule la hauteur de l'arbre a .
- 12 – Écrire une fonction C `arb noeud(int v, arb ag, arb ad)` qui construit un arbre dont la racine a pour étiquette v , dont le fils gauche est a_g et le fils droit est a_d . Dans cette question, il est demandé de se défendre, par le truchement d'une assertion, de toute erreur liée à un échec d'allocation dynamique de mémoire.

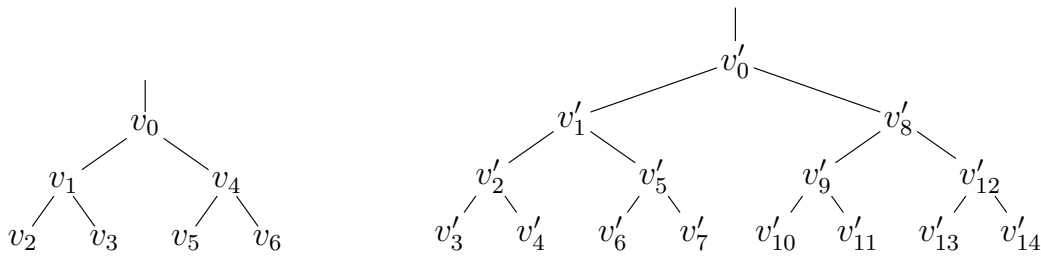


FIGURE 3 – Arbres binaires parfaits de hauteur 2 et de hauteur 3.

2.2. Arbres parfaits

Définition : Un *arbre binaire parfait*, ou simplement *arbre parfait*, est un arbre binaire dont tous les nœuds internes ont exactement deux fils, dont toutes les feuilles sont à la même profondeur et dont les nœuds sont étiquetés par des valeurs entières.

Des exemples d'arbres binaires parfaits sont donnés figure 3.

- 13 – Démontrer que tout arbre binaire parfait de hauteur n possède $2^{n+1} - 1$ nœuds.
- 14 – Écrire une fonction C `bool est_parfait(arb a, int n)` dont la spécification suit :
Précondition : Le pointeur a désigne la racine d'un arbre binaire (dont les nœuds sont bien tous distincts).
Valeur de retour : Booléen `true` si l'arbre pointé est parfait de hauteur n et `false` sinon.
- 15 – Calculer la complexité en temps dans le pire des cas de l'exécution de `est_parfait(a, n)` en fonction de l'entier n .

2.3. Opérations sur les arbres parfaits

Définition : Soient S une structure de données qui permet de stocker une collection d'entiers et t le cardinal de S . Nous disons que la structure de données S est à *accès direct* s'il existe une manière systématique de numéroter chaque élément de S entre 0 et $t - 1$ et s'il existe deux primitives, `acces` et `modif`, qui permettent respectivement de consulter et de modifier n'importe quel élément de S en temps logarithmique en t à partir seulement de son numéro.

Nous souhaitons vérifier qu'un arbre parfait est une structure de données à accès direct. Nous numérotons les éléments d'un arbre parfait en utilisant l'ordre préfixe de gauche à droite de l'arbre (comme illustré dans la figure 3).

- 16 – Écrire une fonction C `arb arb_trouve(arb a, int n, int k)` ainsi spécifiée :
Précondition : Le pointeur a désigne la racine d'un arbre binaire parfait de hauteur n . L'entier k satisfait les inégalités $0 \leq k < 2^{n+1} - 1$.
Valeur de retour : Pointeur vers le k^e nœud de l'arbre dans l'ordre préfixe.

Il est rappelé que la racine d'un arbre est à profondeur 0. Nous donnons la formule de sommation, valable pour tout réel $a \neq 1$ et tout entier t ,

$$\sum_{k=0}^t k \cdot a^k = \frac{((a-1)t-1)a^{t+1} + a}{(a-1)^2}.$$

Première épreuve d'informatique MPI 2023

- 17 – Nous appelons P la profondeur d'un nœud choisi aléatoirement et uniformément parmi les $2^{n+1} - 1$ nœuds d'un arbre binaire parfait de hauteur n . Calculer l'espérance $\mathbb{E}(P)$ de la variable aléatoire P .
- 18 – Déterminer la complexité moyenne en temps de l'instruction `arb_trouve(a, n, k)` lorsque la hauteur n et l'arbre a sont fixés et le numéro k est choisi aléatoirement et uniformément dans l'intervalle $\llbracket 0, 2^{n+1} - 2 \rrbracket$.
- 19 – Écrire une fonction C `int arb_acces(arb a, int n, int k)` qui renvoie la k^{e} valeur de l'arbre a de hauteur n ainsi qu'une fonction C `void arb_modif(arb a, int n, int k, int v)` qui remplace la k^{e} valeur de l'arbre a de hauteur n par la valeur v . Dire finalement si la structure de données arbre binaire parfait est à accès direct.

3. Listes gauches

Les arbres binaires parfaits seuls ne permettent de représenter que des collections d'entiers dont le cardinal est de la forme $2^{n+1} - 1$ où n est entier naturel. Afin de représenter des collections de cardinal quelconque, nous introduisons des suites d'arbres parfaits, aux hauteurs strictement croissantes et savamment choisies.

Définition : Nous appelons *liste binaire gauche de cardinal m sur N arbres* la structure de données constituée de $N + 1$ arbres binaires parfaits $((a_n)_{0 \leq n < N}, e)$ comme suit. Nous décomposons l'entier m selon sa représentation binaire gauche sur N chiffres : $g_{N-1} \dots g_0$. Pour tout indice n compris entre 0 et $N - 1$, si le chiffre g_n n'est pas nul, l'arbre binaire parfait a_n est un arbre de hauteur n , sinon il s'agit de l'arbre vide. Si le chiffre 2 apparaît parmi les chiffres de la représentation gauche de m et si l'indice p est sa position, alors l'arbre e est un arbre binaire parfait de hauteur p , sinon l'arbre e est l'arbre vide. De manière plus courte, nous parlons simplement de *liste gauche*.

Une liste binaire gauche de cardinal m sur N arbres permet de stocker une collection de m éléments entiers. Il apparaît qu'avec N arbres, une liste binaire gauche peut stocker jusqu'à M_N entiers (où l'entier M_N a été introduit à la question 2) et que m est inférieur à M_N .

Indication C : Nous adoptons le type C suivant

```
18. struct ListeGauche {
19.     int hauteur_e;
20.     arb extra;
21.     int nb_arbres;
22.     arb *arbres;
23. };
24. typedef struct ListeGauche lg;
```

Le champ `hauteur_e` contient la hauteur p de l'arbre exceptionnel e . Le champ `extra` désigne la racine de l'arbre exceptionnel e . Le champ `nb_arbres` contient l'entier N . Enfin, le champ `arbres` désigne un tableau de N pointeurs vers les arbres $(a_n)_{0 \leq n < N}$ qui a été alloué dynamiquement.

3.1. Opérations simples sur les listes gauches

□ 20 – Écrire une fonction C `lg lg_init(int N)` qui renvoie une liste gauche de cardinal nul sur N arbres.

□ 21 – Écrire une fonction C `int lg_card(lg l)` qui calcule le cardinal m de la liste gauche ℓ .

Définition : Afin de numéroter l'ensemble des éléments d'une liste gauche $\ell = ((a_n)_{0 \leq n < N}, e)$, nous parcourons d'abord les éléments de l'arbre exceptionnel e dans l'ordre préfixe, puis nous parcourons les éléments des arbres $a_0, a_1, \dots, a_{N-1}$ dans l'ordre préfixe. Les numéros sont attribués aux valeurs rencontrées par ordre de première rencontre.

□ 22 – Écrire une fonction C `arb lg_trouve(lg l, int k)` qui renvoie le k^e nœud de la liste gauche ℓ . On supposera que k est un indice valide ($0 \leq k < m$).

□ 23 – Calculer la complexité en temps dans le pire des cas de `lg_trouve l` en fonction de la capacité maximale M_N de la liste gauche ℓ .

□ 24 – Écrire une fonction C `int lg_acces(lg l, int k)` qui renvoie la k^e valeur de la liste gauche ℓ ainsi qu'une fonction C `void lg_modif(lg l, int k, int v)` qui remplace la k^e valeur de la liste gauche ℓ par la valeur v .

3.2. Ajout et suppression en tête de liste gauche

□ 25 – Soient v une valeur entière et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche de cardinal m , avec $m < M_N$, qui contient les éléments $v_1, \dots, v_m$ dans cet ordre. Décrire, en fonction de la liste gauche ℓ , la liste gauche $\ell' = ((a'_n)_{0 \leq n < N}, e')$ de cardinal $m + 1$ dont les éléments sont $v, v_1, \dots, v_m$ dans cet ordre. En déduire le principe d'une fonction C `bool lg_empile(int v, lg l)` réalisant l'insertion de la valeur v en tête de la liste gauche ℓ où le booléen résultat vaut `true` si l'ajout a eu lieu et vaut `false` si un débordement de capacité de la liste se produit. On ne demande pas le code complet de cette fonction.

□ 26 – Déterminer la complexité en temps dans le pire des cas de la fonction `lg_empile`.

□ 27 – Donner le principe d'une fonction C `bool lg_depille(int *w, lg l)` réalisant le retrait de l'élément de tête de la liste gauche ℓ et son affectation à l'adresse w . Le booléen résultat vaut `true` si le retrait a eu lieu et vaut `false` si l'opération a échoué en raison d'une liste vide. On ne demande pas le code complet.

□ 28 – Donner la complexité en temps dans le pire des cas de la fonction `lg_depille`.

□ 29 – Discuter la possibilité d'obtenir une complexité plus faible à la question 28, quitte à modifier légèrement la définition du type `lg`.

□ 30 – Soit N un entier et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche. Discuter la possibilité de modifier en place et avec une faible complexité en temps la liste gauche ℓ de sorte que le nombre d'arbres N devienne $N + 1$ et que les mêmes éléments demeurent dans la liste.

3.3. Utilisation concurrente des listes gauches

Dans cette sous-section, on raisonne sur les algorithmes en supposant que les fils d'exécution peuvent s'entrelacer mais que les instructions d'un même fil s'exécutent dans l'ordre du programme. L'entête `#include <pthread.h>` a été déclarée ; la syntaxe de certaines fonctions s'y rattachant est rappelée en annexe.

□ 31 – Deux fils d'exécution distincts exécutent la fonction `lg_empile` sur la même liste gauche. Montrer qu'une course critique advient de leur exécution concurrente et que la cohérence de ladite liste gauche n'est pas garantie, autrement dit que certains invariants qui caractérisent la bonne formation d'une liste gauche peuvent être violés à l'issue des exécutions.

Nous nous intéressons finalement au *problème des producteurs et des consommateurs* qui s'échangent des entiers au travers d'un tampon, ici constitué par une unique liste gauche ℓ à N arbres, l'entier N étant fixé à l'avance. Les producteurs écrivent dans la liste gauche ℓ en empilant un entier à la fois en tête, à condition que la liste gauche ℓ ne soit pas pleine ; les consommateurs vident la liste gauche ℓ en dépilant l'entier en tête de la liste, à condition que la liste gauche ℓ ne soit pas vide.

Un seul agent peut accéder au tampon à la fois. Lorsqu'un consommateur souhaite supprimer une donnée alors que le tampon est vide, il est mis en attente ; lorsqu'un producteur souhaite écrire une donnée alors que le tampon est plein, il est mis en attente.

□ 32 – Décrire une solution au problème des producteurs et des consommateurs en s'appuyant sur un verrou et sur deux sémaphores. Détailler sous la forme de code C ou bien de pseudo-code ce que font les producteurs et les consommateurs.

* *

*

Les listes binaires gauches, ou *skew binary lists*, ont été inventées par Eugene Myers en 1983.

A. Rappels de programmation en C

L'expression $1 \ll n$ représente le décalage de la valeur 1 sur n bits : elle a pour valeur l'entier 2^n .

Le type **pthread_t** désigne des fils d'exécution.

L'instruction `pthread_create(pthread_t *th_id, NULL, &ma_fonction, void *args)` crée un nouveau fil d'exécution qui appelle la fonction `ma_fonction` sur le ou les arguments désignés par `args` et qui s'exécute simultanément avec le fil d'exécution appelant.

L'instruction `pthread_join(th_id, NULL)` suspend l'exécution du fil d'exécution appelant jusqu'à ce que le fil d'exécution identifié par `th_id` achève son exécution.

Le type **pthread_mutex_t** désigne des verrous.

L'instruction `pthread_mutex_lock(&v)` verrouille le verrou `v`.

L'instruction `pthread_mutex_unlock(&v)` déverrouille le verrou `v`.

Le type **sem_t** désigne des sémaphores.

L'instruction `sem_init(&s, 0, v)` initialise le sémaphore `s` à la valeur v (avec $v \geq 0$).

L'instruction `sem_wait(&s)` décrémente le compteur du sémaphore `s` : si le compteur est toujours positif, l'appel se termine ; sinon le fil d'exécution appelant est bloqué.

L'instruction `sem_post(&s)` incrémente le compteur du sémaphore `s` et, si le compteur redevient strictement positif, réveille un fil d'exécution bloqué sur `s`.

FIN DE L'ÉPREUVE

A2023 – INFO MPI II

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

Concours Mines-Télécom

CONCOURS 2023

DEUXIÈME ÉPREUVE D'INFORMATIQUE

Durée de l'épreuve : 4 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE II - MPI

L'énoncé de cette épreuve comporte 12 pages de texte.

Cette épreuve concerne uniquement les candidats de la filière MPI.

*Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé,
il le signale sur sa copie et poursuit sa composition en expliquant les raisons
des initiatives qu'il est amené à prendre.*

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.
Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

L'épreuve est composée d'un problème unique, comportant 38 questions. Le problème est divisé en quatre sections qui peuvent être traitées séparément, à condition de lire toutes les définitions de la section 1. Dans la première section (page 1), nous introduisons la *complexité de Kolmogoroff* et étudions des propriétés de calculabilité. Dans la deuxième section (page 3), nous estimons la complexité de Kolmogoroff à l'aide du codage de Huffman. La troisième section (page 4) contient des prolégomènes pour la section suivante. Dans la quatrième section (page 5), nous nous appuyons sur un modèle de calcul épuré, introduit à travers plusieurs langages formels, afin toujours d'estimer la complexité de Kolmogoroff.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n , $\mathcal{D}$ ou π) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`, `d` ou `pi`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Il faudra coder des fonctions à l'aide du langage de programmation OCaml exclusivement, en reprenant l'en-tête de fonctions fourni par le sujet, sans s'obliger à recopier la déclaration des types. Il est permis d'utiliser la totalité du langage OCaml mais il est recommandé de s'en tenir aux fonctions les plus courantes afin de rester compréhensible. Des rappels ponctuels de documentation du langage OCaml peuvent être proposés à titre d'aide. Quand l'énoncé demande de coder une fonction, sauf demande explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle.

1 Complexité de Kolmogoroff

Nous notons Σ l'ensemble ordonné des 256 caractères ASCII étendus usuels et Σ^* l'ensemble des chaînes de caractères. Pour toute chaîne de caractères $x \in \Sigma^*$, la longueur de x , notée $|x|$, est le nombre de caractères qui la composent. Par exemple, la longueur de la chaîne "abac" est 4. Le nombre d'occurrences d'un symbole $\sigma \in \Sigma$ dans une chaîne de caractères $x \in \Sigma^*$ est noté $|x|_\sigma$. Par exemple, $|abac|_a = 2$.

Dans l'ensemble du sujet, nous nous appuyons sur une *machine universelle*, c'est-à-dire une fonction OCaml `eval`, de type `string -> string`, telle que :

- si la chaîne x , de type `string`, est le code source d'une expression OCaml y de type `string` et si l'exécution du code x se termine sans erreur, alors `eval x` se termine et a pour valeur de retour la valeur de y ;
- sinon, `eval x` ne se termine pas.

Les exécutions de `eval` ont lieu sur une machine idéale dont la mémoire est infinie et qui est capable de gérer des types natifs de taille quelconque.

Seconde épreuve d'informatique MPI 2023

Définition : Pour toute chaîne de caractères $y \in \Sigma^*$, nous disons que la chaîne de caractères $x \in \Sigma^*$ est une *description* de la chaîne y si le calcul `eval x` se termine et renvoie la chaîne y . Nous appelons *complexité de Kolmogoroff* de y et notons $K(y)$ la longueur de la plus courte chaîne de caractères $x \in \Sigma^*$ qui décrit y .

L'objet de ce sujet est d'étudier des propriétés et diverses majorations de la complexité de Kolmogoroff. Dans toutes nos illustrations, nous nous concentrons sur la description de la chaîne de caractères

$$y_0 = "1000000\dots" \in \Sigma^*,$$

qui correspond à l'entier $10^{(10^{10})}$ écrit en base 10 et que nous fixons une fois pour toutes.

1.1 Un premier exemple

□ 1 – Proposer une première majoration de la complexité de Kolmogoroff $K(y_0)$, qui s'appuie sur la chaîne de caractères $x_0 = y_0 = "1000000\dots"$ comme description de y_0 .

Indication OCaml : Il est rappelé que la fonction `string_of_int` convertit un entier en une chaîne de caractères.

□ 2 – Soient n un entier naturel et n' la partie entière de $\frac{n}{2}$. Exprimer la quantité 10^n en fonction de $10^{n'}$. Compléter le code OCaml suivant en utilisant une stratégie « diviser pour régner » :

```
let exp10 n = (* Calcul de 10^n a ecrire *)
in string_of_int (exp10 (exp10 10))
```

afin d'en faire une description de la chaîne de caractères $y_0 = "1000000\dots"$. En déduire une nouvelle borne grossière (à 10^2 près) de la complexité de Kolmogoroff $K(y_0)$, significativement meilleure que celle de la question 1.

□ 3 – Décrire quelle ou quelles difficultés adviendraient si l'on exécutait le code de la question 2 sur une machine réelle.

1.2 Quelques propriétés

Indication OCaml : L'expression `String.make (n : int) (sigma : char) : string` désigne la chaîne de caractères répétant n fois le caractère $\sigma \in \Sigma$. Pour tout entier n compris entre 0 et 255, `Char.chr (n : int) : char` désigne le n^{e} caractère dans la numérotation ASCII.

□ 4 – Présenter une bijection $\varphi : \mathbb{N} \rightarrow \Sigma^*$ entre l'ensemble des entiers naturels et l'ensemble de chaînes de caractères. En écrire le code sous la forme d'une fonction OCaml `phi (n : int) : string`.

Seconde épreuve d'informatique MPI 2023

Nous prétendons avoir écrit une fonction OCaml `kolmogoroff` (`y : string`) : `int` qui calcule la complexité de Kolmogoroff $K(y)$.

□ 5 – Écrire une fonction OCaml `psi` (`m : int`) : `int` dont la valeur de retour est l'entier

$$\psi(m) = \min \{n \in \mathbb{N}; K(\varphi(n)) \geq m\}$$

où $\varphi : \mathbb{N} \rightarrow \Sigma^*$ est la bijection définie à la question 4 et qui utilise la fonction `kolmogoroff`.

□ 6 – Établir d'une part que, pour tout entier naturel m , on a

$$K(\varphi(\psi(m))) \geq m$$

et d'autre part que l'on a

$$K(\varphi(\psi(m))) = O(\log m).$$

Discuter l'existence de la fonction OCaml `kolmogoroff`.

Définition : Nous appelons *décompresseur* toute fonction OCaml `d : string -> string`. Pour toute chaîne de caractères $y \in \Sigma^*$ et pour tout décompresseur $\mathcal{D}$ (noté informatiquement `d`), nous disons que la chaîne de caractères z est une *description* de la chaîne y par rapport à $\mathcal{D}$ si le calcul `eval (d z)` se termine sans erreur et a pour valeur de retour y . Nous appelons *complexité de Kolmogoroff par rapport à $\mathcal{D}$* de la chaîne y , et notons $K_{\mathcal{D}}(y)$, la longueur de la plus courte chaîne de caractères $z \in \Sigma^*$ qui décrit y par rapport à $\mathcal{D}$.

□ 7 – Dire comment se nomme en informatique un programme qui transforme un code source dans un certain langage de programmation en un code équivalent dans un second langage.

□ 8 – Montrer que pour tout décompresseur $\mathcal{D}$, il existe une constante entière $c_{\mathcal{D}}$ telle que, pour toute chaîne de caractères y , nous avons :

$$K(y) \leq K_{\mathcal{D}}(y) + c_{\mathcal{D}}.$$

2 Estimation de la complexité grâce au décompresseur de Huffman

Nous fixons dans cette section la chaîne de caractères $x_0 \in \Sigma^*$ suivante

```
"let rec t e=if e=0 then 1 else let n=e-1 in 10*t n in let n=t 10 in t n".
```

La chaîne x_0 contient 71 caractères, l'espace étant un caractère et les guillemets ne faisant pas partie de la chaîne.

□ 9 – Inférer le type OCaml de l'expression dénotée par la chaîne de caractères x_0 .

Seconde épreuve d'informatique MPI 2023

- 10 – Déterminer la valeur de l'évaluation de x_0 en tant que code source OCaml.
- 11 – Signaler une ou plusieurs caractéristiques du code source x_0 qui rend la lecture de ce code impénétrable par un humain.

Indication OCaml : Nous rappelons le détail de quelques fonctions du module OCaml `Hashtbl` permettant de manipuler des dictionnaires (ou tableaux associatifs) mutables.

- `Hashtbl.create (n : int) : ('a, 'b) Hashtbl.t` crée un dictionnaire vide de taille initiale n .
- `Hashtbl.add (d : ('a, 'b) Hashtbl.t) (k : 'a) (v : 'b) : unit` ajoute une association entre la clé k et la valeur v au dictionnaire d .
- `Hashtbl.find_opt (d : ('a, 'b) Hashtbl.t) (k : 'a) : 'b option` vaut `Some v` si le dictionnaire d possède une association entre la clé k et la valeur v et vaut `None` sinon.

- 12 – Écrire une fonction OCaml `count (x : string) : (char, int) Hashtbl.t` dont la valeur de retour est un dictionnaire qui associe chaque caractère $\sigma \in \Sigma$ présent dans la chaîne x à son nombre d'occurrences $|x|_\sigma$.

Nous exécutons `count x0` et obtenons le dictionnaire suivant :

'n'	't'	'i'	'l'	'1'	'c'	'f'	'h'	'r'	's'	'-'	'*'	'0'	'='	'e'	' '
8	8	4	4	4	1	1	1	1	1	1	1	3	4	10	19

Nous appelons z_0 la chaîne de bits correspondant à la compression de la chaîne x_0 par l'algorithme de Huffman.

- 13 – Dessiner un arbre de Huffman associé à la chaîne de caractères x_0 . Il est recommandé de placer les feuilles de gauche à droite comme dans le tableau ci-dessus.

Nous notons $\mathcal{H}$ le décompresseur qui utilise l'arbre de Huffman de la question 13 pour transformer une chaîne de bits $z \in \{0, 1\}^*$ en un mot $x \in \Sigma^*$ et renvoie la chaîne de caractères `"string_of_int (x)"`.

- 14 – Calculer la longueur $|z_0|$ de la chaîne obtenue après compression de x_0 . En déduire que la complexité de Kolmogoroff de $y_0 = 10000\dots$ par rapport au décompresseur $\mathcal{H}$ vérifie

$$K_{\mathcal{H}}(y_0) \leq 239.$$

3 Interlude

Nous souhaitons écrire une fonction `new_string : unit -> string` ainsi spécifiée : à chaque appel, une chaîne de caractères inédite est produite. Nous offrons trois propositions de code.

Seconde épreuve d'informatique MPI 2023

```

let inc s = s := "a" ^ (!s); !s    (* Commun aux 3 propositions *)

let new_string1 =                  (* Proposition 1 *)
  let seed1 = ref "#" in
  fun () -> inc seed1

let new_string2 () =              (* Proposition 2 *)
  let seed2 = ref "#" in
  inc seed2

let seed3 = ref "#"              (* Proposition 3 *)
let new_string3 () =
  inc seed3

```

- 15 – Analyser la portée de la variable `seed1`, respectivement `seed2` et `seed3`, dans la fonction `new_string1`, respectivement `new_string2` et `new_string3`.
- 16 – Dédire de la question 15 laquelle des trois fonctions `new_string1`, `new_string2` et `new_string3` ne respecte pas la spécification. Écrire un test qui permet de discriminer la fonction erronée.
- 17 – Dédire de la question 15 laquelle des deux fonctions correctes restantes est plus propice à des erreurs de manipulation. Expliquer.

4 Estimation de la complexité grâce au décompresseur de De Bruijn

Nous nous avisons que la complexité de Kolmogoroff est influencée par la verbosité et l'expressivité du langage de programmation. Dans cette section, nous tentons de réduire l'encombrement ou la facilité d'écriture dus à la syntaxe du langage OCaml en introduisant un modèle de calcul nouveau et épuré.

4.1 Construction syntaxique d'un langage

Nous présentons systématiquement les *grammaires* sous la forme d'un quadruplet (N, Γ, S, Π) où N désigne l'alphabet des symboles non terminaux, Γ l'alphabet des symboles terminaux, $S \in N$ le symbole initial et Π l'ensemble des règles de production, de la forme $X \rightarrow \gamma$ avec $X \in N$ et $\gamma \in (N \cup \Gamma)^*$.

Une *dérivation immédiate* se note $\alpha \Rightarrow \beta$, avec $(\alpha, \beta) \in ((N \cup \Gamma)^*)^2$, et indique qu'il existe une règle de production $X \rightarrow \gamma$ de Π et des décompositions $\alpha = \alpha_1 X \alpha_2$ et $\beta = \alpha_1 \gamma \alpha_2$, avec $(\alpha_1, \alpha_2) \in ((N \cup \Gamma)^*)^2$. Une *dérivation* se note $\Rightarrow^*$ et indique l'existence d'une suite finie, éventuellement vide, de dérivations immédiates. Enfin, le *langage engendré par une grammaire* $\mathcal{G}$ se note $L(\mathcal{G})$ et désigne l'ensemble des mots de Γ^* qui dérivent du symbole initial S .

Seconde épreuve d'informatique MPI 2023

Selon les questions, nous représentons les mots de Γ par le type `char list` ou le type `string`.

Soit $\mathcal{G}_0$ la grammaire $(\{V\}, \{a, b, \#\}, V, \Pi_0)$ dont les règles de production sont

$$\Pi_0 : \quad V \rightarrow aV \mid bV \mid \#.$$

□ 18 – Exhiber une expression régulière qui dénote le langage $L(\mathcal{G}_0)$.

□ 19 – Dessiner l'automate de Glushkov associé à l'expression régulière de la question 18.

□ 20 – Écrire une fonction OCaml `parseV (w : char list) : string * char list` dont la spécification suit :

Pré-condition : Il existe une décomposition du mot w en $w = vs$ avec $v \in L(\mathcal{G}_0)$ et $s \in \Sigma^*$.

Valeur de retour : Couple (v, s) où le préfixe v est représenté par le type `string` et s est le suffixe restant.

Effet : Une exception `SyntaxError` est levée si la pré-condition n'est pas satisfaite.

Soit $\mathcal{G}_1$ la grammaire $(\{T\}, \{(_, _)\}, T, \Pi_1)$ dont les règles de production sont

$$\Pi_1 : \quad T \rightarrow (TT) \mid _.$$

□ 21 – Montrer que le langage $L(\mathcal{G}_1)$ est sans préfixe, c'est-à-dire qu'il n'existe pas deux mots non vides w et w' dans $L(\mathcal{G}_1)$ tels que w soit un préfixe strict de w' . On pourra, par exemple, raisonner par induction structurale sur le nombre de parenthèses ouvrantes et fermantes qui se trouvent dans un préfixe d'un mot de $L(\mathcal{G}_1)$.

□ 22 – Montrer que la grammaire $\mathcal{G}_1$ n'est pas ambiguë, c'est-à-dire que, pour tout mot du langage $L(\mathcal{G}_1)$, il n'existe qu'un seul arbre d'analyse (parfois aussi appelé arbre de dérivation) associé.

Soit $\mathcal{G}_2$ la grammaire $(\{T\}, \{\text{var}, (_, _), ->\}, T, \Pi_2)$ dont les règles de production sont

$$\Pi_2 : \quad T \rightarrow \text{var} \mid (TT) \mid \text{var} \rightarrow T.$$

□ 23 – Montrer que la grammaire $\mathcal{G}_2$ n'est pas ambiguë.

Soit finalement $\mathcal{G}$ la grammaire $(\{T, V\}, \{a, b, \#, (_, _), ->\}, T, \Pi)$ dont les règles de production sont

$$\Pi : \quad \begin{cases} T \rightarrow V \mid (TT) \mid V \rightarrow T \\ V \rightarrow aV \mid bV \mid \# \end{cases}$$

Nous admettons que la grammaire $\mathcal{G}$ n'est pas ambiguë. Nous appelons *variable* les mots de $L(\mathcal{G}_0)$. Informellement, la grammaire $\mathcal{G}$ engendre un langage $L(\mathcal{G})$ qui permet de parler de *variables*, d'*applications* d'une expression à une autre et de *fonctions*. Il nous reste à en construire une sémantique, ce qui sera fait en section 4.3.

Seconde épreuve d'informatique MPI 2023

Afin de représenter l'arbre d'analyse d'un mot du langage $L(\mathcal{G})$, nous introduisons le type `ada` par la déclaration suivante.

```
type ada = V of string | A of (ada * ada) | F of (string * ada)
```

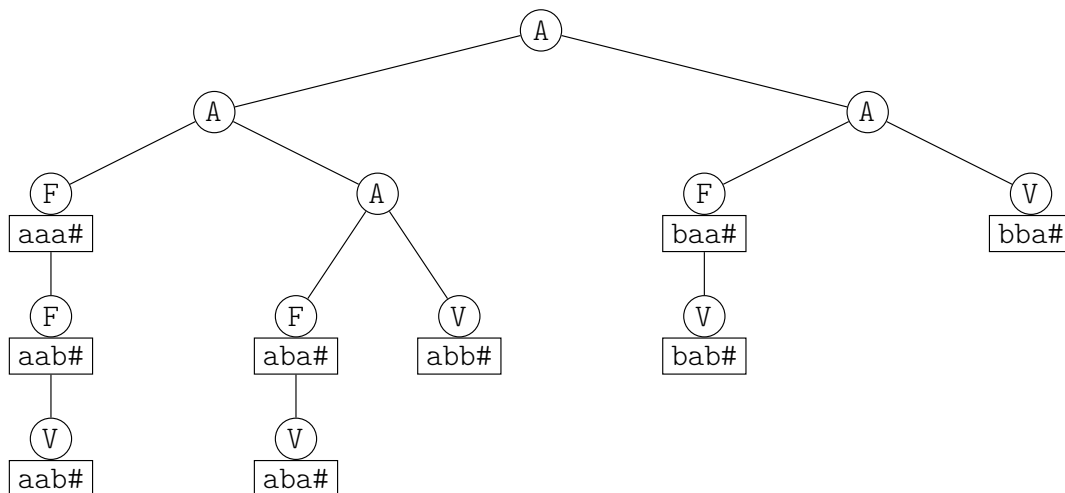
Nous notons $\mathcal{A}$ l'ensemble des arbres d'analyse.

Le constructeur OCaml `V` représente les dérivations immédiates de règle $T \rightarrow V$ et introduit les feuilles ; le constructeur OCaml `A` représente les dérivations immédiates de règle $T \rightarrow (TT)$ et introduit des nœuds internes d'arité 2 ; le constructeur OCaml `F` représente les dérivations immédiates de règle $V \rightarrow T$ et introduit des nœuds internes d'arité 1. Les dérivations à partir du symbole non terminal V sont directement représentées par une valeur OCaml de type `string`.

Par exemple, le mot

$$((aaa\# \rightarrow aab\# \rightarrow aab\#(aba\# \rightarrow aba\#abb\#))(baa\# \rightarrow bab\#bba\#)) \in L(\mathcal{G})$$

admet pour arbre d'analyse :



□ 24 – Écrire une fonction OCaml `parseT (w : char list) : ada * char list` dont la spécification suit :

Pré-condition : Il existe une décomposition du mot w en $w = ps$ avec $p \in L(\mathcal{G})$ et $s \in \Sigma^*$, dans laquelle le préfixe p est de longueur maximale.

Valeur de retour : Couple (a, s) où a est l'arbre d'analyse de p et s est le suffixe restant.

Effet : Une exception `SyntaxError` est levée si la pré-condition n'est pas satisfaite.

Indication OCaml : Nous supposons définie une fonction `charlist_of_string : string -> char list` qui transforme une chaîne de caractères en une liste de caractères.

□ 25 – Écrire une fonction OCaml `parse (w : string) : ada` dont la valeur de retour est l'unique arbre d'analyse de w quand $w \in L(\mathcal{G})$ et qui lève une exception `SyntaxError` sinon.

Seconde épreuve d'informatique MPI 2023

Définition : Pour tout entier naturel $n \in \mathbb{N}$, nous définissons le mot

$$[n] = b\# \rightarrow a\# \rightarrow (b\#(b\# \dots (b\#(b\#a\#)) \dots)) \in L(\mathcal{G})$$

où la variable $b\#$ apparaît n fois à droite des deux symboles $\rightarrow$.

□ 26 – Indiquer s'il existe un automate fini capable de reconnaître le langage $\{[n]; n \in \mathbb{N}\}$ et justifier la réponse.

Définition : Plus généralement, nous disons qu'un mot w de $L(\mathcal{G})$ *incarne* un entier naturel $n \in \mathbb{N}$ s'il existe deux variables distinctes v_1 et v_2 dans $L(\mathcal{G}_0)$ telles que w est de la forme

$$w = v_2 \rightarrow v_1 \rightarrow (v_2(v_2 \dots (v_2(v_2v_1)) \dots)) \in L(\mathcal{G})$$

où la variable v_2 apparaît n fois à droite des deux symboles $\rightarrow$.

□ 27 – Écrire une fonction OCaml `int_of_ada (a : ada) : int` dont la valeur de retour est l'entier naturel n incarné par a . Si un tel entier n'existe pas, une exception `SyntaxError` est levée.

4.2 Réécriture des variables et sérialisation

□ 28 – Nommer une structure de donnée concrète efficace qui réalise le type de donnée abstrait « ensemble » lorsqu'il existe une relation d'ordre entre les objets à ranger.

Indication OCaml : Nous supposons définis un module OCaml `StringSet` permettant de construire des ensembles de chaînes de caractères persistants, de type `StringSet.t`, et des fonctions :

- `StringSet.mem : string -> StringSet.t -> bool` qui teste l'appartenance d'un élément à un ensemble.
- `StringSet.remove : string -> StringSet.t -> StringSet.t` qui retourne un ensemble privé d'un élément.
- `StringSet.singleton : string -> StringSet.t` qui construit un singleton à partir d'un élément.
- `StringSet.union : StringSet.t -> StringSet.t -> StringSet.t` qui construit l'union de deux ensembles.

Définition : L'ensemble des *variables libres* $VL(a) \subseteq \Sigma^*$ d'un arbre d'analyse $a \in \mathcal{A}$ est défini par induction structurelle avec les règles d'inférence suivantes :

- si l'arbre d'analyse a est de la forme $V \ v$, où $v \in L(\mathcal{G}_0)$, alors $VL(a)$ est le singleton $\{v\}$;
- si l'arbre d'analyse a est de la forme $A \ (a_1, a_2)$, où a_1 et a_2 sont deux arbres d'analyse, alors $VL(a)$ est la réunion $VL(a_1) \cup VL(a_2)$;
- si l'arbre d'analyse a est de la forme $F \ (v, a_1)$, où $v \in L(\mathcal{G}_0)$ et a_1 est un arbre d'analyse, alors $VL(a)$ est l'ensemble $VL(a_1) \setminus \{v\}$.

□ 29 – Écrire une fonction OCaml `free_vars (a : ada) : StringSet.t` dont la valeur de retour est l'ensemble des variables libres de l'arbre d'analyse a .

Seconde épreuve d'informatique MPI 2023

Définition : Un arbre d'analyse est dit *clos* s'il ne contient pas de variables libres ; de même, un mot de $L(\mathcal{G})$ est dit *clos* si son arbre d'analyse est clos.

Dans un arbre d'analyse clos $a \in \mathcal{A}$, pour toute chaîne de caractères $v \in L(\mathcal{G}_0)$, si la construction OCaml $V\ v$ apparaît comme feuille de l'arbre a , alors il existe un nœud interne de la forme $F\ (v, _)$ parmi les ascendants de $V\ v$ qui coïncide avec l'« introduction » de la variable v .

Afin de ne plus s'encombrer avec des variables nommées par une chaîne de caractères, nous adoptons une nouvelle représentation des mots du langage $L(\mathcal{G})$ sous forme d'un arbre, appelé *terme de De Bruijn*.

Définition : Un terme de De Bruijn s'obtient à partir d'un arbre d'analyse $a \in \mathcal{A}$ en remplaçant toute feuille de l'arbre a , disons $V\ v$ avec $v \in L(\mathcal{G}_0)$, par une feuille étiquetée par l'entier $u = 1 + \ell$, où $\ell \in \mathbb{N}$ est le nombre de nœuds internes de la forme $F\ (v', _)$ avec $v' \in L(\mathcal{G}_0) \setminus \{v\}$, qui existent entre ladite feuille $V\ v$ et le plus proche nœud interne de la forme $F\ (v, _)$ parmi ses ascendants dans l'arbre d'analyse a .

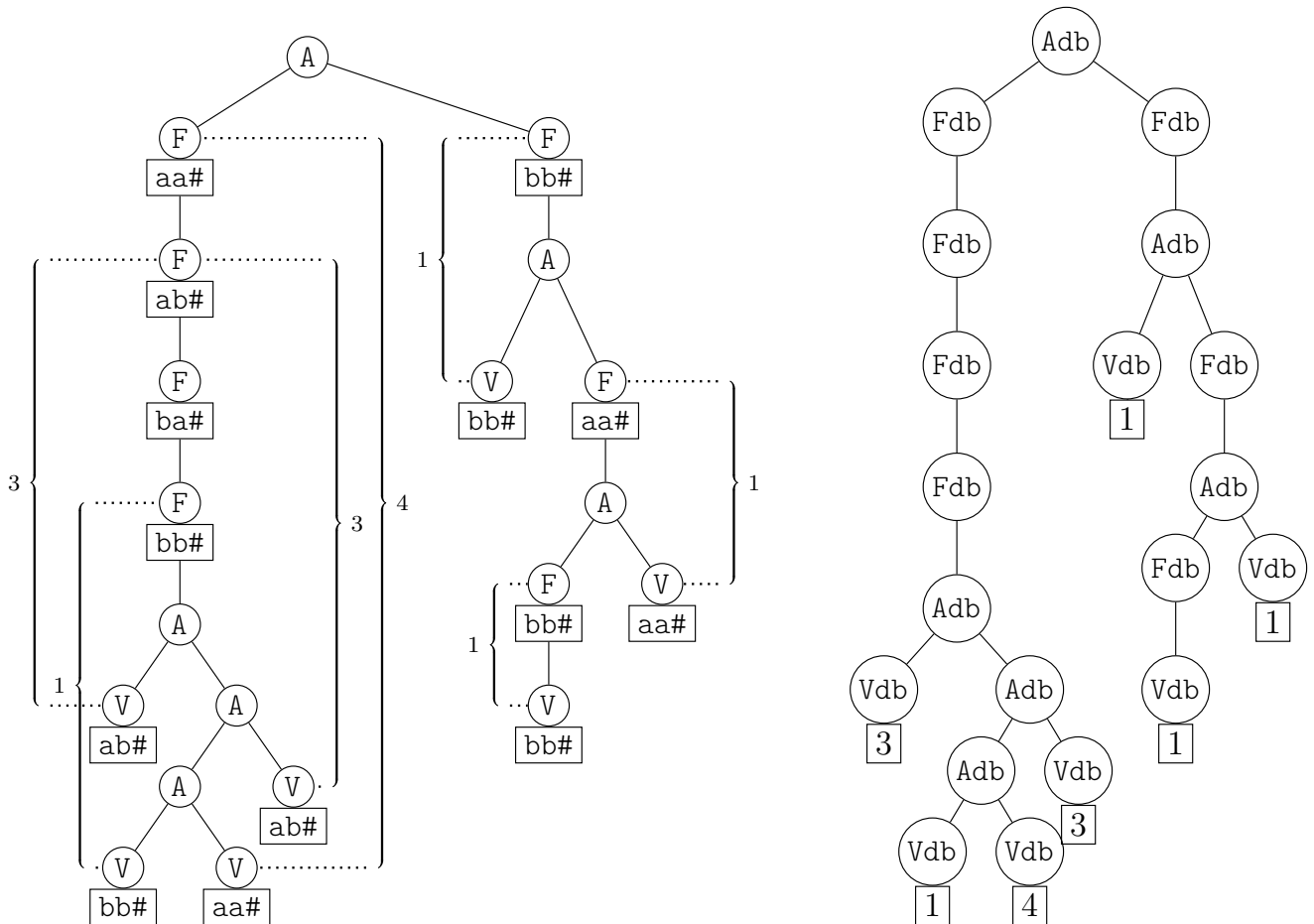
Nous déclarons un nouveau type

type tdb = Vdb **of** int | Adb **of** (tdb * tdb) | Fdb **of** tdb

Nous notons $\mathcal{T}$ l'ensemble des termes de De Bruijn.

Voici un exemple de construction d'un terme de De Bruijn (à droite) à partir d'un arbre d'analyse (à gauche) du mot

(aa#->ab#->ba#->bb#->(ab#((bb#aa#)ab#))bb#->(bb#aa#->(bb#->bb#aa#))) :



Seconde épreuve d'informatique MPI 2023

□ 30 – Dessiner le terme de De Bruijn qui représente le mot $\lceil n \rceil$.

□ 31 – Écrire une fonction OCaml `ada_of_tdb (t : tdb) : ada` dont la valeur de retour est un arbre d'analyse clos associé au terme de De Bruijn t .

Définition : Le *codage binaire* des termes de De Bruijn est l'application $t \in \mathcal{T} \mapsto \hat{t} \in \{0, 1\}^*$ définie par induction structurelle avec les règles d'inférence suivantes sur l'ensemble des termes de De Bruijn :

- si le terme $t \in \mathcal{T}$ est de la forme $\text{Vdb } u$, avec $u \in \mathbb{N}^*$, alors $\hat{t}$ est la chaîne de caractères $11 \dots 10$ (avec le symbole 1 répété u fois).
- si le terme $t \in \mathcal{T}$ est de la forme $\text{Adb } (t_1, t_2)$, t_1 et t_2 étant deux termes de De Bruijn, alors $\hat{t}$ est la chaîne de caractères $01\hat{t}_1\hat{t}_2$.
- si le terme $t \in \mathcal{T}$ est de la forme $\text{Fdb } t_1$, où t_1 est un terme de De Bruijn, alors $\hat{t}$ est la chaîne de caractères $00\hat{t}_1$.

□ 32 – Soient n un entier naturel et t le terme de De Bruijn associé au mot $\lceil n \rceil$ et obtenu à la question 30. Calculer la longueur $|\hat{t}|$ de la chaîne de caractères qui encode t .

□ 33 – Vérifier que le codage binaire des termes de De Bruijn est une application injective.

Nous utilisons le type `string` avec les caractères '0' et '1' afin de représenter des codages binaires de termes de De Bruijn.

□ 34 – Écrire une fonction OCaml `decode (z : string) : tdb` dont la valeur de retour est l'unique terme de De Bruijn t tel que $\hat{t} = z$ si un tel terme t existe et qui lève une exception `SyntaxError` sinon.

Il est possible de s'appuyer sur la fonction `charlist_of_string` précédemment présentée.

4.3 Interpréteur et décompresseur de De Bruijn

Dans cette sous-section, nous construisons un interpréteur, c'est-à-dire une procédure qui évalue les expressions appartenant au langage $L(\mathcal{G})$ et en déduisons une nouvelle description du mot $y_0 = "1000000\dots"$ relative à un décompresseur approprié.

Naïvement, lorsque nous rencontrons un sous-mot de la forme $w = (v \rightarrow w_b \ w_a)$ avec $v \in L(\mathcal{G}_0)$ et $(w_a, w_b) \in (L(\mathcal{G}))^2$, nous aimerions que w soit équivalent à un mot tiré de w_b dont les occurrences de la variable v ont été remplacées par w_a . Autrement dit, lorsqu'un arbre d'analyse est de la forme $A \ (F \ (v, b), a)$, nous voudrions construire un nouvel arbre à partir de $b \in \mathcal{A}$ et dans lequel les apparitions de $v \in L(\mathcal{G}_0)$ sont devenues des sous-arbres $a \in \mathcal{A}$.

Seconde épreuve d'informatique MPI 2023

Définition : Soient $(a, b) \in (\mathcal{A})^2$ deux arbres d'analyse et $v \in L(\mathcal{G}_0)$ une variable. La *substitution de la variable v par l'arbre a dans l'arbre b* est l'application $[v \leftarrow a] : \mathcal{A} \rightarrow \mathcal{A}$ définie par induction structurale avec les règles d'inférence suivantes :

- Si l'arbre $b \in \mathcal{A}$ est de la forme $V \ v_1$, avec $v_1 \in L(\mathcal{G}_0)$, alors

$$[v \leftarrow a](b) = \begin{cases} a & \text{si } v = v_1 \\ b & \text{si } v \neq v_1. \end{cases}$$

- Si l'arbre $b \in \mathcal{A}$ est de la forme $A \ (b_1, b_2)$, où $b_1 \in \mathcal{A}$ et $b_2 \in \mathcal{A}$ sont deux arbres d'analyse, alors

$$[v \leftarrow a](b) = A \ (b_1', b_2')$$

où $b_1' = [v \leftarrow a](b_1)$ et $b_2' = [v \leftarrow a](b_2)$.

- Si l'arbre $b \in \mathcal{A}$ est de la forme $F \ (v_1, b_1)$, avec $v_1 \in L(\mathcal{G}_0)$ et $b_1 \in \mathcal{A}$, alors

$$[v \leftarrow a](b) = \begin{cases} b & \text{si } v = v_1 \\ F \ (v_1, b_1') & \text{avec } b_1' = [v \leftarrow a](b_1) \quad \text{si } v \neq v_1 \text{ et } v_1 \notin VL(a) \\ F \ (v_1', b_1') & \text{avec } b_1' = [v \leftarrow a]([v_1 \leftarrow v_1'](b_1)) \quad \text{sinon} \end{cases}$$

où $v_1' \in L(\mathcal{G}_0)$ est une variable inédite qui n'appartient pas à $VL(a) \cup VL(b) \cup \{v, v_1\}$.

Nous supposons déjà programmée une fonction `new_string : unit -> string` inspirée de la section 3 qui permet, si besoin, d'engendrer une variable de $L(\mathcal{G}_0)$ jamais encore utilisée.

□ 35 – Écrire une fonction OCaml `substitute (v : string) (by_a : ada) (in_b : ada) : ada` qui substitue la variable v par l'arbre a dans l'arbre b .

Définition : La *réduction en un pas* est l'application $\triangleright : \mathcal{A} \rightarrow \mathcal{A}$ définie par induction structurale avec les règles d'inférence suivantes :

- Si l'arbre $a \in \mathcal{A}$ est de la forme $V \ v$, où $v \in L(\mathcal{G}_0)$, alors la valeur $\triangleright(a)$ n'est pas définie.
- Si l'arbre $a \in \mathcal{A}$ est de la forme $A \ (a_1, a_2)$, où $a_1 \in \mathcal{A}$ et $a_2 \in \mathcal{A}$ sont deux arbres d'analyse, alors

$$\triangleright(a) = \begin{cases} A \ (a_1, a_2') & \text{avec } a_2' = \triangleright(a_2) & \text{si } a_1 \text{ est de la forme } V \ v \\ a' & \text{avec } a' = [v \leftarrow a_2](a_{11}) & \text{si } a_1 \text{ est de la forme } F \ (v, a_{11}) \\ A \ (a_1', a_2) & \text{avec } a_1' = \triangleright(a_1) & \text{sinon.} \end{cases}$$

- Si l'arbre $a \in \mathcal{A}$ est de la forme $F \ (v, a_1)$, où $v \in L(\mathcal{G}_0)$ et $a_1 \in \mathcal{A}$, alors

$$\triangleright(a) = F \ (v, a_1') \text{ où } a_1' = \triangleright(a_1).$$

□ 36 – Écrire une fonction OCaml `reduce_one_step (a : ada) : ada` qui implémente la réduction en un pas $\triangleright$. Lorsque `reduce_one_step` rencontre un cas non défini, une exception `NoReduction` est levée.

□ 37 – Écrire une fonction OCaml `interpret (a : ada) : ada` qui applique répétitivement la réduction en un pas $\triangleright$ à l'arbre d'analyse a jusqu'à ce que la réduction ne soit plus définie. La valeur de retour est le dernier arbre d'analyse rencontré.

 Seconde épreuve d'informatique MPI 2023

Nous admettons que, pour tout entier naturel non nul n , si π est le mot de $L(\mathcal{G})$

$$\pi = (a\# \rightarrow ((a\#a\#)a\#) \lceil n \rceil),$$

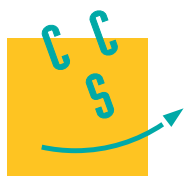
l'expression `interpret (parse pi)` produit une incarnation de l'entier $n^{(n^n)}$. Nous notons $\mathcal{B}$ le décompresseur défini par

```
let decompB (z : string) : string =
  "string_of_int␣(int_of_ada␣(interpret␣(ada_of_tdb␣(decode␣" ^ z ^ "))))"
```

□ 38 – Proposer une méthode pour obtenir une chaîne de caractères z_0 , formée uniquement de 0 et de 1, telle que `decompB z0` a pour valeur de retour la chaîne de caractères $y_0 = "1000000\dots"$ et en calculer la longueur. En déduire que la complexité de Kolmogoroff de y_0 par rapport au décompresseur $\mathcal{B}$ vérifie

$$K_{\mathcal{B}}(y_0) \leq 70.$$

FIN DE L'ÉPREUVE



CONCOURS CENTRALE•SUPÉLEC

Informatique

MPI

2023

4 heures

Calculatrice autorisée

Le sujet est composé de deux parties indépendantes, la première utilise le langage C et la seconde le langage OCaml. Toutes deux traitent de graphes.

La première se concentre sur un problème classique, le problème du voyageur de commerce. Elle étudie d'abord sa complexité, en le comparant notamment au problème très proche du cycle et du chemin hamiltonien, puis en le réduisant à 3-SAT, et en donnant un résultat sur l'approximabilité du problème. On propose ensuite une heuristique, l'algorithme de Christofides, qui repose sur différentes notions : arbres couvrants, couplages, circuit eulérien. On montrera que, dans une variante du problème où les distances vérifient l'inégalité triangulaire, l'algorithme de Christofides est une approximation de facteur $3/2$. Dans cette première partie, les graphes seront représentés par des matrices d'adjacence en C.

La deuxième partie s'intéresse à des processus d'édition sur des arbres non racinés dont les feuilles sont étiquetées, qui sont notamment utilisés en bio-informatique pour représenter l'évolution des espèces. On étudiera en particulier l'espace induit sur ces arbres par ces processus d'édition et on représentera cet espace par un graphe. On montrera notamment que le graphe induit par l'un de ces processus d'édition possède un cycle hamiltonien. On utilisera pour cette partie le langage OCaml pour implémenter des arbres par une définition récursive et des graphes par liste d'adjacence.

I Problème du voyageur de commerce

On considère des graphes non orientés $G(V, E)$ où V est l'ensemble des sommets et E l'ensemble des arêtes. On notera $n = |V|$ le nombre de sommets.

Un *chemin* est une suite de sommets reliés par des arêtes. On dit qu'un chemin *pass*e par un sommet si ce sommet appartient au chemin. Un *circuit* est un chemin qui commence et se termine au même sommet. Un *chemin hamiltonien* est un chemin qui passe une et une seule fois par chaque sommet du graphe. Un *circuit hamiltonien* est un circuit qui passe par chaque sommet une et une seule fois.

Le *problème du voyageur de commerce* consiste, étant donnée une liste de villes toutes reliées entre elles, à trouver le circuit le plus court qui passe une et une seule fois par chacune des villes.

Plus formellement, on considère un graphe complet non orienté, dont les arêtes sont étiquetées avec des nombres entiers strictement positifs, appelés *poids*, et on cherche le circuit passant par chacun des sommets du graphe qui minimise la somme des poids des arêtes. On appellera *poids d'un circuit* la somme des poids des arêtes empruntées par ce circuit. Une solution au problème du voyageur de commerce est un circuit hamiltonien de poids minimal.

Dans cette partie, on représente les graphes en C par des matrices d'adjacence. Le poids d'une arête est représenté par un `int`, une arête absente étant représentée par un 0. On représente un graphe par une structure de données avec deux attributs : son nombre de sommets `V` et un pointeur `adj` vers sa matrice d'adjacence de taille $V \times V$. Les sommets sont associés aux entiers de 0 à $V-1$.

```
struct Graphe {
    int V;
    int* adj;
};
```

Pour accéder au poids de l'arête entre le sommet `i` et le sommet `j` du graphe `G`, on pourra utiliser l'expression `G.adj[i * G.V + j]`.

On pourra par la suite utiliser les deux fonctions suivantes, qui sont supposées travailler en temps constant :

```
struct Graphe alloue_graphe(int V) {
    int* adj = malloc(V * V * sizeof(int));
    struct Graphe g = {.V = V, .adj = adj};
    return g;
}

void libere_graphe(struct Graphe g) {
    free(g.adj);
}
```

La fonction `alloue_graphe` prend comme argument un nombre V et renvoie un graphe qui possède V sommets et dont la matrice d'adjacence est allouée sur le tas, grâce à la fonction `malloc`. La fonction `libere_graphe` libère la mémoire du tas occupée par la matrice d'adjacence d'un graphe dont on n'a plus besoin.

On définit également une structure `Chemin` qui représente un chemin par un attribut `longueur` et un attribut `l_sommets`, pointeur vers un tableau à `longueur` éléments. Si C est un chemin et k un entier naturel strictement plus petit que $C.\text{longueur}$, alors $C.l_sommets[k]$ est le k -ième sommet du chemin C . De même que pour les graphes, on définit aussi des fonctions permettant d'allouer un chemin et de libérer un chemin dont on n'a plus besoin, et qui sont supposées travailler en temps constant.

```
struct Chemin {
    int longueur;
    int* l_sommets;
};

struct Chemin alloue_chemin(int longueur) {
    int* l_sommets = malloc(longueur * sizeof(int));
    struct Chemin c = {.longueur = longueur, .l_sommets = l_sommets};
    return c;
}

void libere_chemin(struct Chemin c) {
    free(c.l_sommets);
}
```

I.A – Prise en main du problème et appartenance à NP

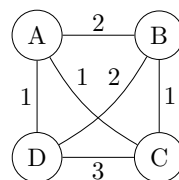


Figure 1 Exemple d'instance du problème du voyageur de commerce

Q 1. Donner une solution du problème du voyageur de commerce sur l'exemple de la figure 1 en précisant le poids du circuit trouvé.

Q 2. Donner le nombre de circuits hamiltoniens sur un graphe complet de n sommets. Donner un exemple de pondération pour que chacun de ces circuits ait un poids minimal pour le problème du voyageur de commerce.

Q 3. Écrire une fonction

```
int poids_chemin(struct Graphe g, struct Chemin c);
```

qui prend en arguments un graphe et un chemin et renvoie le poids de ce chemin. Donner la complexité de cette fonction.

Q 4. Le problème du voyageur de commerce est un problème d'optimisation ; à l'aide d'un seuil, transformer ce problème en un problème de décision. Montrer que ce nouveau problème, que nous appellerons par la suite « problème de décision du voyageur de commerce », appartient à la classe de complexité NP.

I.B – Étude de la complexité

Le *problème du chemin hamiltonien* consiste, étant donné un graphe non orienté et deux sommets a et b , à déterminer s'il existe un chemin hamiltonien commençant en a et finissant en b . Le *problème du chemin hamiltonien orienté* consiste, étant donné un graphe orienté et deux sommets a et b , à déterminer s'il existe un chemin hamiltonien commençant en a et finissant en b . Le *problème du circuit hamiltonien* consiste, étant donné un graphe non orienté, à déterminer s'il existe un circuit hamiltonien dans ce graphe.

I.B.1) NP-complétude

Q 5. Montrer que le problème du chemin hamiltonien se réduit au problème du circuit hamiltonien.

Q 6. Montrer que le problème du circuit hamiltonien se réduit au problème de décision du voyageur de commerce.

Q 7. Montrer que le problème du chemin hamiltonien orienté se réduit au problème du chemin hamiltonien.

Le problème 3-SAT consiste à déterminer la satisfiabilité d'une formule logique sous forme normale conjonctive avec exactement 3 littéraux : pour n clauses C_i et m variables y_k , déterminer s'il existe une valuation des y_k qui permette de rendre vraie la formule $C_1 \wedge C_2 \wedge \dots \wedge C_n$ avec $C_i = x_{i,1} \vee x_{i,2} \vee x_{i,3}$ sachant que, pour chaque i et chaque p , il existe un k tel que $x_{i,p} = y_k$ ou $x_{i,p} = \neg y_k$. On admet que le problème 3-SAT est NP-complet.

On va maintenant montrer que 3-SAT se réduit au problème du chemin hamiltonien orienté.

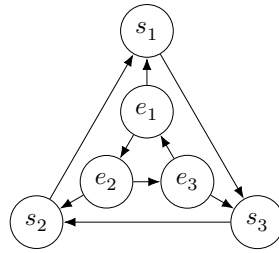


Figure 2 Graphe A

Q 8. On considère le graphe A (figure 2). Montrer qu'il existe un chemin entrant par e_1 et passant par tous les sommets pour ressortir en s_1 . Puis qu'il existe un chemin entrant par e_1 et sortant par s_1 et un chemin entrant par e_2 et sortant par s_2 , tels que chaque sommet soit visité par un et un seul des deux chemins. Même question, mais avec trois chemins, pour e_1, e_2, e_3 et s_1, s_2, s_3 .

On se donne une instance du problème 3-SAT, pour n clauses C_i et m variables y_k : $C_1 \wedge C_2 \wedge \dots \wedge C_n$ avec $C_i = x_{i,1} \vee x_{i,2} \vee x_{i,3}$ et $x_{i,p} = y_k$ ou $x_{i,p} = \neg y_k$. On veut donc savoir s'il existe une valuation des y_k pour que la formule soit vraie.

On construit alors le graphe orienté G de la manière suivante :

- pour chaque variable y_k on crée un sommet v_k ;
- on ajoute un sommet supplémentaire v_{m+1} ;
- pour chaque clause C_i on ajoute une copie du graphe A , notée A_i ;
- pour chaque variable y_k , on note $C_{k_1}, \dots, C_{k_\ell}$ les clauses dans lesquelles y_k apparait en positif. On relie alors v_k à A_{k_1} par un arc allant de v_k vers e_p dans A_{k_1} lorsque y_k est en position p dans C_{k_1} c'est-à-dire lorsque $C_{k_1} = x_{k_1,1} \vee x_{k_1,2} \vee x_{k_1,3}$ et $x_{k_1,p} = y_k$. On relie ensuite la sortie s_p de A_{k_1} à l'entrée de A_{k_2} correspondant à la position de y_k dans C_{k_2} et ainsi de suite jusqu'au dernier dont on relie la sortie à v_{k+1} . On appelle G_k^+ le sous-graphe constitué du sommet v_k , des graphes $A_{k_1}, A_{k_2}, \dots, A_{k_\ell}$ et du sommet v_{k+1} , ainsi que des arcs que l'on vient d'ajouter entre eux en considérant y_k et les clauses dans lesquelles il apparait positivement ;
- on crée de même des arcs pour chaque variable y_k et chaque clause dans laquelle y_k apparait en négatif. On note G_k^- le sous-graphe correspondant entre v_k et v_{k+1} .

Q 9. Montrer que pour toute valuation de la formule il existe un chemin hamiltonien orienté de v_1 à v_{m+1} dans le graphe G .

Q 10. Montrer, en une dizaine de lignes au maximum, que pour chaque chemin hamiltonien orienté de v_1 à v_{m+1} il existe bien une valuation.

Q 11. En déduire que le problème du circuit hamiltonien et le problème de décision du voyageur de commerce sont NP-complets.

I.B.2) Approximation

Soit $\varepsilon > 0$, on va montrer que, si $P \neq NP$, il n'existe pas de $1 + \varepsilon$ approximation pour le problème du voyageur de commerce. Un algorithme est une $1 + \varepsilon$ approximation à un problème d'optimisation d'un poids p lorsque la solution proposée de poids p se compare toujours à la solution optimale p^* par $p < (1 + \varepsilon)p^*$.

Soit G un graphe non orienté à n sommets, on considère le graphe complet G' , de mêmes sommets que G , avec des poids aux arêtes obtenus en donnant un poids de 1 aux arêtes de G et un poids de $n(1 + \varepsilon) + 1$ aux arêtes qui ne sont pas dans G .

Q 12. Montrer que si G possède un circuit hamiltonien, alors G' possède un circuit de poids n .

Q 13. Montrer que si G ne possède pas de circuit hamiltonien alors toute solution pour l'instance de voyageur de commerce est de poids au moins $n(2 + \varepsilon)$.

Q 14. En déduire que, si $P \neq NP$, il n'existe pas de $1 + \varepsilon$ approximation au problème du voyageur de commerce.

I.C – Algorithme de Christofides

On va proposer une heuristique pour le problème du voyageur de commerce, l'algorithme de Christofides, et on va montrer que, sous certaines conditions sur le graphe en entrée, cette heuristique constitue un algorithme d'approximation. L'algorithme prend en argument un graphe G et procède comme suit :

- calculer un arbre couvrant de poids minimal T de G ;
- en notant I l'ensemble des sommets de degré impair dans T , calculer un couplage parfait M de poids minimum dans le sous-graphe de G induit par les sommets de I , $G|_I$;
- construire H le multigraphe ayant pour sommet les sommets de G et comme arêtes les arêtes de M et celles de T ;

- trouver un cycle eulérien dans H ;
- transformer le cycle eulérien en circuit hamiltonien en supprimant les éventuels sommets vus plusieurs fois.

Dans la suite, on étudie plus précisément certaines étapes de cet algorithme, avant de proposer une implémentation de cet algorithme.

I.C.1) Arbre couvrant

Un arbre couvrant est un sous-graphe connexe sans cycle d'un graphe avec les mêmes sommets. On appelle poids de l'arbre la somme des poids des arêtes de cet arbre. On rappelle que l'algorithme de Kruskal est un algorithme glouton qui vise à construire un arbre couvrant de poids minimal en considérant les arêtes par poids croissant et en ajoutant chaque arête si elle ne crée pas de cycle.

Pour cela, on va représenter une arête par une structure de données avec trois attributs : **s1** et **s2** donnent les sommets reliés par l'arête et **p** son poids :

```
struct Arete {
    int s1;
    int s2;
    int p;
};
```

Q 15. Écrire une fonction

```
struct Arete* liste_aretes(struct Graphe g);
```

qui prend en argument un graphe complet g et alloue et renvoie un tableau contenant les $\frac{g \cdot V \times (g \cdot V - 1)}{2}$ arêtes du graphe.

On dispose d'une fonction

```
void tri_aretes(struct Arete a[], int k);
```

qui prend en arguments un tableau d'arêtes et sa longueur et trie le tableau par ordre croissant de poids. La complexité de cette fonction est en $\mathcal{O}(k \ln(k))$.

Q 16. Écrire une fonction

```
struct Graphe kruskal(struct Graphe g);
```

implémentant l'algorithme de Kruskal qui renvoie un graphe représentant l'arbre couvrant de poids minimal du graphe donné en argument. On mettra des 0 lorsque l'arête est absente et des 1 lorsqu'elle est présente. Donner la complexité de la fonction `kruskal`.

Q 17. Montrer la correction de cet algorithme, c'est-à-dire l'optimalité de la solution proposée pour le problème d'arbre couvrant de poids minimal.

I.C.2) Couplage

On appelle couplage d'un graphe un ensemble d'arêtes qui n'ont pas de sommets en commun. Un couplage est parfait si tous les sommets du graphe appartiennent à une arête du couplage.

Q 18. Écrire une fonction

```
int degre(struct Graphe g, int i);
```

qui prend en arguments un graphe et l'indice d'un sommet et renvoie le degré de ce sommet.

Q 19. Écrire une fonction

```
int* sommets_impairs(struct Graphe g, int* nb_sommets);
```

qui prend en arguments un graphe g et un pointeur vers un entier. Cette fonction alloue sur le tas un tableau, le remplit avec les numéros des sommets de degré impair et le renvoie. Par ailleurs, elle renseigne l'entier pointé par `nb_sommets` avec le nombre des sommets de degré impair.

Q 20. Montrer l'existence d'un couplage parfait de poids minimal dans $G|_I$.

On dispose des deux fonctions suivantes :

```
struct Graphe graphe_induit(struct Graphe g, int nb_sommets, int* liste_sommets);
struct Graphe couplage(struct Graphe g);
```

La fonction `graphe_induit` renvoie le graphe induit dans un graphe g donné par un nombre et un tableau de sommets comme ceux de la question 19.

La fonction `couplage` renvoie un couplage parfait de poids minimal s'il existe, sous la forme d'un graphe représentant ce couplage, avec des 0 lorsque l'arête est absente et 1 lorsque l'arête est présente.

On supposera ces deux fonctions de complexité polynomiale.

I.C.3) Cycle eulérien

On appelle cycle eulérien un circuit qui passe par chaque arête une et une unique fois. On admet qu'un graphe possède un cycle eulérien si et seulement si les degrés des sommets de ce graphe sont pairs et que le graphe est connexe. Un multigraphe est un graphe dans lequel il peut exister plusieurs arêtes reliant un même couple de sommets.

Q 21. Montrer que le multigraphe H , défini dans l'introduction de la sous-partie I.C, possède un cycle eulérien.

On dispose des trois fonctions suivantes :

```
struct Multigraphe multigraphe(struct Graphe g1, struct Graphe g2);
struct Chemin eulerien(struct Multigraphe h);
void libere_multigraphe(struct Multigraphe h);
```

La fonction `multigraphe` prend en arguments deux graphes sur les mêmes sommets et renvoie le multigraphe obtenu en considérant les arêtes des deux graphes. La fonction `eulerien` renvoie un circuit eulérien d'un multigraphe sous la forme d'un chemin. La fonction `libere_multigraphe` libère la mémoire du tas utilisée par un multigraphe. Toutes trois sont supposées de complexité polynomiale.

Q 22. Écrire une fonction

```
struct Chemin euler_to_hamilton(struct Chemin c);
```

qui transforme un cycle eulérien c du multigraphe H en un circuit hamiltonien du graphe G en supprimant les doublons, et renvoie le chemin représentant la suite des sommets.

I.C.4) Implémentation

Q 23. Sur l'exemple de la figure 1, réaliser les différentes étapes de l'algorithme. On ne demande pas de détailler les étapes pour trouver un couplage et un cycle eulérien.

Q 24. Écrire une fonction

```
struct Chemin christofides(struct Graphe g);
```

qui implémente l'algorithme de Christofides, en prenant soin de libérer la mémoire allouée sur le tas qui n'est plus utilisée.

Q 25. Justifier que la fonction `christofides` renvoie bien un circuit hamiltonien.

Q 26. Montrer que la fonction `christofides` est de complexité polynomiale.

I.C.5) Preuve de l'approximation

On va maintenant montrer que cet algorithme est une $3/2$ -approximation pour le problème du voyageur de commerce, dans le cas où les poids des arêtes vérifient l'inégalité triangulaire, c'est-à-dire que pour tout sommet u, v, w les poids des arêtes vérifient, en notant $c_{i,j}$ le poids de l'arête entre des sommets i et j : $c_{u,v} \leq c_{u,w} + c_{w,v}$.

On note U une solution optimale et $c(U)$ son poids.

Q 27. Montrer que $c(T) \leq c(U)$, où $c(T)$ est le coût de l'arbre couvrant minimal.

Q 28. Montrer que $c(M) \leq 0.5c(U)$ où $c(M)$ est le coût du couplage parfait minimal.

Q 29. Montrer que la solution construite par l'algorithme est une $3/2$ -approximation de U .

Q 30. Sans la supposition de l'inégalité triangulaire, cette solution est-elle toujours une $3/2$ -approximation ? Proposer un contre-exemple ou une justification.

II Espaces d'arbres

Dans cette partie on considère des arbres binaires non racinés avec des feuilles étiquetées. Un *arbre binaire non raciné* est un graphe connexe sans cycle dont les nœuds sont de degrés 1 ou 3. Un *arbre binaire raciné* est un graphe connexe sans cycle dont les nœuds sont de degrés 1 ou 3 sauf exactement un nœud qui est de degré 2. Les nœuds de degré 1 sont appelés des *feuilles*, les nœuds de degré 2 ou 3 des *nœuds internes* et l'unique nœud de degré 2 d'un arbre binaire raciné est appelé sa *racine*. Un graphe réduit à un unique nœud est considéré comme un arbre raciné dont le nœud est à la fois feuille et racine. Les arêtes d'un arbre sont appelées des *branches*, les branches reliant des sommets de degré 2 ou 3 *branches internes*. On étiquette ensuite les feuilles d'un arbre à n feuilles par des entiers distincts entre 1 et n . On s'intéresse à des opérations d'édition qui transforment un arbre à n feuilles en un second arbre à n feuilles.

La première opération, notée SPR, pour *Subtree Prune and Regraft* (ou découpe et greffe d'un sous-arbre), prend en entrée 2 branches e et f de l'arbre. La branche e est supprimée, créant deux arbres racinés par les sommets à chaque extrémité de e , notons ces arbres T_1 (qui contient la branche f) et T_2 (qui ne la contient pas). On divise ensuite la branche f pour créer un nouveau nœud v de degré 2. Le sous-arbre raciné T_2 est alors rattaché par sa racine à v par une nouvelle branche ; v devient alors de degré 3. On obtient alors un arbre raciné, avec une racine de degré 2, que l'on supprime en reliant directement ses deux voisins pour obtenir un arbre non raciné.

On s'intéresse également à une autre opération, notée NNI, pour *Nearest Neighbor Interchange* (ou échange entre plus proches voisins). Cette opération prend une branche interne de l'arbre e et échange entre eux deux des quatre sous-arbres incidents à cette branche. C'est-à-dire, en notant a et b les extrémités de la branche interne e , a a deux sommets voisins qui ne sont pas b , a_1 et a_2 , et de même pour b , b_1 et b_2 . On choisit alors un des a_i et un des b_j à échanger, par exemple a_1 et b_1 , on supprime l'arête reliant a à a_1 et b à b_1 et on relie a à b_1 et b à a_1 .

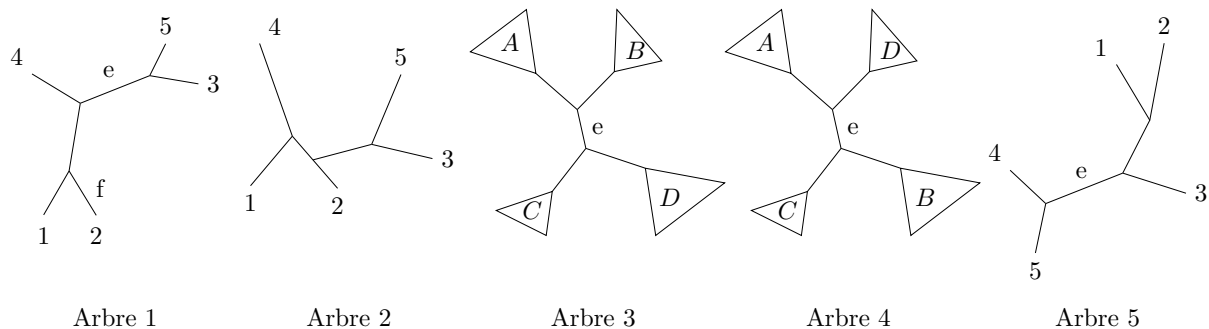


Figure 3 Exemples d'arbres non racinés

La figure 3 présente quelques exemples d'arbres non racinés :

- l'arbre 1 est un arbre binaire non raciné avec ses feuilles étiquetées ;
- l'arbre 2 est obtenu à partir de l'arbre 1 par un SPR en coupant la branche e et en la greffant sur la branche f ;
- dans l'arbre 3, où les triangles représentent des sous-arbres racinés, la branche interne e sépare 4 sous-arbres racinés A, B, C et D ;
- l'arbre 4 est obtenu à partir de l'arbre 3 par un mouvement NNI sur la branche e en échangeant les sous-arbres B et D ;
- l'arbre 5 est obtenu à partir de l'arbre 1 par un NNI sur la branche e en échangeant le sous-arbre contenant la feuille 5 et le sous-arbre contenant les feuilles 1 et 2.

On étudie l'espace des arbres binaires non racinés étiquetés de n feuilles $B(n)$ suivant ces deux processus d'édition. On définit $G_{\text{NNI}}(n)$ le graphe dont les sommets sont les arbres de $B(n)$ et dans lequel une arête relie deux arbres si l'on peut passer de l'un à l'autre par un mouvement NNI. On définit de même $G_{\text{SPR}}(n)$ avec les mouvements SPR. On va notamment s'intéresser à la structure de $G_{\text{NNI}}(5)$, écrire un programme permettant de construire $G_{\text{NNI}}(n)$ et montrer que $G_{\text{SPR}}(n)$ est hamiltonien, c'est-à-dire qu'il possède un circuit hamiltonien.

II.A – Prise en main

Q 31. Montrer que l'on peut passer de l'arbre 1 à l'arbre 2 par une opération NNI. Dessiner tous les voisins de l'arbre 1 dans $G_{\text{NNI}}(5)$.

Q 32. Donner le nombre de branches et de nœuds d'un arbre de $B(n)$, pour $n \geq 2$.

Q 33. On note $RB(n)$ l'ensemble des arbres binaires racinés à n feuilles étiquetées. Montrer que $|RB(n)| = (2n - 3)|B(n)|$ et que $|B(n)| = |RB(n - 1)|$. En déduire le nombre d'arbres binaires $B(n)$ en fonction de n .

Q 34. Tracer $G_{\text{NNI}}(4)$.

Q 35. Combien de voisins un arbre de $B(n)$ a-t-il dans $G_{\text{NNI}}(n)$?

On appelle arbre chenille un arbre binaire non raciné dans lequel il existe un chemin passant une et une seule fois par chaque nœud interne.

Q 36. Montrer que l'on peut passer de tout arbre de $B(n)$ à un arbre chenille, en effectuant seulement des mouvements NNI. En déduire que $G_{\text{NNI}}(n)$ est connexe.

Q 37. Montrer que tout mouvement SPR peut se décomposer en mouvements NNI, et que tous les mouvements NNI sont des mouvements SPR. En déduire que $G_{\text{SPR}}(n)$ est connexe.

II.B – Étude de $G_{\text{NNI}}(5)$

Q 38. Combien de sommets possède $G_{\text{NNI}}(5)$ et quel est le degré de ces sommets ?

Q 39. Montrer que tout arbre de $G_{\text{NNI}}(5)$ fait partie de cycles de longueur 5, de deux cycles de longueurs 3, et que pour tout arbre deux arbres sont à distance 3 de cet arbre.

Q 40. Tracer $G_{\text{NNI}}(5)$.

II.C – Construction de $G_{\text{NNI}}(n)$

On va écrire en OCaml un programme qui construit $G_{\text{NNI}}(n)$. Pour représenter les arbres binaires (racinés ou non), nous allons utiliser la structure de données suivante :

```
type arbre = Feuille of int
           | Noeud of arbre * arbre ;;
```

Un arbre raciné est alors représenté par sa racine, qui peut être soit une feuille soit un nœud interne ayant un sous-arbre gauche et un sous-arbre droit.

Un arbre non raciné à n feuilles est représenté à partir d'une racine fictive ajoutée entre la feuille d'étiquette n et le nœud auquel elle est reliée.

Enfin, pour assurer l'unicité de la représentation informatique d'un arbre, et ainsi simplifier les programmes, on adopte la convention, pour chaque nœud interne, que la plus grande des étiquettes du sous-arbre droit est supérieure aux étiquettes du sous-arbre gauche.

Ainsi, l'arbre 1 de la figure 3 est représenté par :

```
Noeud (Noeud (Feuille 3,
              Noeud (Noeud (Feuille 1,
                            Feuille 2),
                            Feuille 4)),
      Feuille 5);;
```

On représente les graphes par liste d'adjacence, plus précisément par une liste de couples dont le premier élément est le nom d'un sommet et le second la liste d'adjacence de ce sommet.

```
type graphe = (arbre * arbre list) list;;
```

Q 41. Écrire une fonction **feuilles** qui prend en argument un arbre et renvoie la liste des étiquettes de ses feuilles.

Q 42. Écrire une fonction **degres** qui prend en argument un graphe implémenté par liste d'adjacence et renvoie la liste des degrés de ces nœuds.

Q 43. Écrire une fonction **egaux** qui teste si deux arbres sont égaux, au sens des arbres binaires non racinés étiquetés. Justifier que cette fonction est correcte.

Q 44. Étant donnés une liste d'arbres et un arbre, écrire une fonction **appartient** qui teste si l'arbre fait partie de la liste.

Q 45. En remarquant que parmi les 4 sous-arbres à échanger pour un mouvement de NNI autour d'une branche interne, on peut en choisir un qui restera fixe, écrire une fonction **voisinsNNI** qui prend en argument un arbre et renvoie tous les arbres que l'on peut obtenir à partir de celui-ci par un mouvement NNI.

Q 46. Écrire une fonction **chenille** qui prend en argument un entier n et renvoie l'arbre chenille dans lequel les feuilles sont rangées de 1 à n .

Q 47. Écrire une fonction **insere** qui prend en arguments un arbre et un graphe et ajoute l'arbre à la liste des sommets du graphe.

Q 48. Écrire une fonction **relie** qui prend en arguments un graphe et deux arbres et qui rajoute au graphe une arête reliant les deux sommets, si elle n'est pas déjà présente.

Q 49. Écrire une fonction **grapheNNI** qui prend en argument un entier n et renvoie $G_{\text{NNI}}(n)$, en implémentant le graphe avec des listes d'adjacences. Justifier la correction et la terminaison de votre programme.

II.D – $G_{\text{SPR}}(n)$ est hamiltonien

On va démontrer par récurrence que $G_{\text{SPR}}(n)$ est hamiltonien, c'est-à-dire qu'il possède un circuit hamiltonien, comme défini en première partie.

On note S_i l'ensemble des arbres de taille $n + 1$ obtenu à partir d'un arbre t_i de taille n en ajoutant une feuille étiquetée $n + 1$ à une des branches de t_i . Pour ajouter une feuille à une branche, on supprime la branche et on crée un nouveau nœud, relié à la nouvelle feuille et aux deux nœuds qui étaient reliés par la branche.

Q 50. Montrer que les sous-graphes $G_{\text{SPR}}(n + 1)_{|S_i}$, induits dans $G_{\text{SPR}}(n + 1)$ par les sommets de S_i sont complets.

Q 51. Soient t_i et t_j deux sommets de $G_{\text{SPR}}(n)$ reliés entre eux, montrer qu'il existe des arêtes entre S_i et S_j dans $G_{\text{SPR}}(n + 1)$.

Q 52. Démontrer par récurrence que $G_{\text{SPR}}(n)$ est hamiltonien.

• • • FIN • • •

SESSION 2023



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI**

INFORMATIQUE**Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
 - Ne pas utiliser de correcteur.
 - Écrire le mot *FIN* à la fin de votre composition.
-

Les calculatrices sont interdites.

Le sujet est composé de trois parties, toutes indépendantes.

Partie I - Palindromes

En langue française, "ressasser" est le mot palindrome le plus long, tandis qu'il semble que "saippua-kauppias" soit le plus long mot palindrome au monde, désignant un marchand de savon en Finlande. L'objet de cette partie est de compter le nombre de palindromes présents dans un mot donné.

Soit Σ un alphabet fini contenant au moins deux lettres. On note $u = u_0 \cdots u_{n-1}$ un *mot* sur Σ , composé de n lettres $u_i \in \Sigma, i \in \llbracket 0, n-1 \rrbracket$. La longueur de u est notée $|u|$. Pour $0 \leq i < j \leq n$, on note $u[i, j]$ le mot $u_i \cdots u_{j-1}$. L'ensemble des mots construits sur Σ et contenant le mot vide ϵ est noté Σ^* .

Définition 1 (Miroir)

Soit $u \in \Sigma^*$. Le *miroir* de $u = u_0 \cdots u_{n-1}$, noté $\bar{u}$, est le mot $\bar{u} = u_{n-1} \cdots u_0$. Par convention $\bar{\epsilon} = \epsilon$.

Définition 2 (Palindrome)

$u \in \Sigma^*$ est un *palindrome* si et seulement si $u = \bar{u}$.

Par convention, le mot vide ϵ n'est pas considéré comme un palindrome.

On dira qu'un palindrome u est *pair* (respectivement *impair*) lorsque sa longueur $|u|$ est paire (resp. impaire).

On recherche donc dans cette partie le nombre de palindromes facteurs d'un mot $u \in \Sigma^*$ (comptés avec les multiplicités éventuelles), soit le cardinal de l'ensemble $\{(i, j), 0 \leq i < j \leq |u|, u[i, j] = \overline{u[i, j]}\}$. Dit autrement, on recherche le nombre de palindromes contenus dans u .

Q1. Si $\Sigma = \{a, b\}$, donner le nombre de palindromes contenus dans le mot $u = babb$.

En parcourant naïvement les lettres d'un mot u donné, on peut proposer un algorithme en pseudo-code permettant de compter le nombre palindromes de u (**algorithme 1**).

Algorithme 1 - Décompte naïf du nombre de palindromes contenus dans un mot donné.

Entrées : Un mot u .

Sorties : Le nombre de palindromes contenus dans u .

début

```

    nb = 0
    pour i=0 à |u|-1 faire
        pour j=0 à |u|-1 faire
            estPalindrome=True
            pour k=i à j-1 faire
                si u[i] ≠ u[j-k-1] alors
                    estPalindrome=False
                Sortir du pour k
            si estPalindrome alors
                nb = nb+1
    retourner nb
```

Q2. Évaluer la complexité au pire des cas de l'**algorithme 1** en fonction de $|u|$.

On souhaite bien sûr améliorer cette première idée. Pour ce faire, on utilise tout d'abord le paradigme de la programmation dynamique.

Pour $u \in \Sigma^*$, on définit un tableau de booléens P de taille $(|u|+1) \times (|u|+1)$, $P[i][j]$ étant vrai si $u[i, j]$ est un palindrome. On a donc pour tout $i \in \llbracket 0, |u|-1 \rrbracket$, $P[i][i+1] = \text{True}$.

Q3. Soit $u[i, j]$ un mot. À quelles conditions sur u_i , u_{j-1} et $u[i+1, j-1]$ le mot $u[i, j]$ est-il un palindrome ?

Q4. En déduire une relation de récurrence vérifiée par les coefficients de P .

Q5. Écrire un algorithme de programmation dynamique en pseudo-code résolvant le problème. Évaluer sa complexité.

On insère maintenant entre chaque paire de lettres de u , ainsi qu'au début et à la fin du mot, un symbole spécial noté $\#$. On appelle ce nouveau mot $u^\#$. Ainsi, le mot $u = abba$ se transforme en $u^\# = \#a\#b\#b\#a\#$.

Q6. Montrer que les palindromes de $u^\#$, s'ils existent, sont tous impairs.

Q7. Soit v un palindrome de longueur $2k + 1$ de u , $k \in \mathbb{N}$. On construit le mot $u^\#$. Donner les deux palindromes correspondants à v dans $u^\#$.

Q8. Même question si v est un palindrome de longueur $2k$ de u .

Q9. En déduire une stratégie de recherche de tous les palindromes de u .

On voit que les palindromes impairs sont importants. On va donc construire un algorithme de recherche de ce type de palindrome.

Définition 3 (Rayon d'un palindrome)

Soient $u \in \Sigma^*$ et $i \in \llbracket 0, |u| - 1 \rrbracket$. On dit qu'il existe un *palindrome centré en i de rayon $\rho > 0$* si :

- (i). $i - \rho \geq 0$,
- (ii). $i + \rho + 1 \leq |u|$,
- (iii). $u[i - \rho, i + \rho + 1]$ est un palindrome.

Le *rayon maximal* $\hat{\rho}_i$ d'un palindrome centré en i est le plus grand rayon d'un palindrome centré en i .

Par exemple, si $u = abbababab$ et $i = 4$, alors b est un palindrome centré en 4 de rayon 0, aba est un palindrome centré en 4 de rayon 1, $babab$ est un palindrome centré en 4 de rayon 2 et c'est le plus grand, donc $\hat{\rho}_4 = 2$.

Q10. En remarquant qu'un palindrome impair est centré sur une lettre (ou un symbole spécial dans le cas de $u^\#$), proposer un algorithme en pseudo-code permettant de compter le nombre de palindromes impairs d'un mot u . Évaluer sa complexité.

Q11. Soient $u \in \Sigma^*$, $i \in \llbracket 0, |u| - 1 \rrbracket$ et $0 < j \in \llbracket 1, \hat{\rho}_i \rrbracket$. En exploitant les différentes symétries, montrer qu'il existe un palindrome centré en $i + j$ de rayon $\min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. En déduire $\hat{\rho}_{i+j} \geq \min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. Préciser à quelle condition il y a égalité.

En utilisant cette remarque, on développe un algorithme, dit de Manacher, qui construit un tableau d'entiers T permettant de compter le nombre de palindromes d'un mot u . Plus précisément, pour chaque position $i \in \llbracket 1, |u| - 1 \rrbracket$, $T[i]$ indique le rayon maximal $\hat{\rho}_i$, donc tel que la sous-chaîne de $i - \rho_i$ à $i - \rho_i + 1$ est un palindrome. L'algorithme 2 consiste à incrémenter $T[i]$ jusqu'à trouver le plus grand palindrome $u[i - T[i], i + T[i] + 1]$ centré en i .

Algorithme 2 - Algorithme de Manacher

Entrées : Un mot u

Sorties : Un tableau T .

début

Initialiser un tableau T de $|u|$ cases, initialisées à 0

$k=0$

pour $i = 1$ **à** $|u| - 1$ **faire**

$j = i - k$

si $T[k] \geq j$ **alors**

$T[i] = \min(T[k - j], T[k] - j)$

tant que $(i - (T[i] + 1)) \geq 0$ **ET** $(i + T[i] + 1) < |u|$ **ET** $(u[i - (T[i] + 1)] = u[i + T[i] + 1])$ **faire**

$T[i] = T[i] + 1$

$k = i$

retourner T

Q12. Comment trouver à partir de cet algorithme le nombre de palindromes de u ?

Q13. Quelle est la complexité de cet algorithme ? Justifier.

Partie II - Traversée de rivière

Cette partie comporte des questions nécessitant un **code OCaml**.

Dans une vallée des Alpes, un passage à gué fait de cailloux permet de traverser la rivière. Deux groupes de randonneurs arrivent simultanément sur les berges gauche et droite de cette rivière et veulent la traverser. Le chemin étant très étroit, une seule personne peut se trouver sur chaque caillou de ce chemin (**figure 1**). Un randonneur sur la berge de gauche peut avancer d'un caillou (vers la droite sur la **figure 1**) et sauter par dessus le randonneur devant lui (un caillou à droite) si le caillou où il atterit est libre. De même, chaque randonneur de la berge de droite peut avancer d'un caillou (vers la gauche sur la **figure 1**) et sauter par dessus le randonneur devant lui, dans la mesure où le caillou sur lequel il atterit est libre. Une fois engagés, les randonneurs ne peuvent pas faire marche arrière. De plus, pour simplifier, on suppose qu'une fois tous les randonneurs sur le chemin, il ne reste qu'un caillou de libre.



Figure 1 - Les randonneurs et le chemin de cailloux

Le chemin de cailloux est défini par un tableau d'entiers :

```
type chemin_caillou = int array
```

Dans ce tableau, un randonneur venant de la berge de gauche est représenté par un 1, un randonneur issu de la berge de droite par un 2 et un caillou libre par un 0.

- Q14.** Écrire une fonction de signature `caillou_vide : chemin_caillou -> int` qui détermine la position du caillou inoccupé.
- Q15.** Écrire une fonction de signature `echange : chemin_caillou -> int -> int -> chemin_caillou` qui permute les valeurs codées sur deux cailloux. Le tableau d'entiers initial représentant le chemin n'est pas modifié. On pourra utiliser ici la fonction `copy` du module `Array`.
- Q16.** Écrire une fonction de signature `randonneurG_avance : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse avancer (vers la droite).
- Q17.** Écrire une fonction de signature `randonneurG_saute : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse sauter (vers la droite) au-dessus d'un randonneur.

On supposera dans la suite les fonctions de signature `randonneurD_avance : chemin_caillou -> bool` et `randonneurD_saute : chemin_caillou -> bool` écrites de manière similaire pour les randonneurs venant de la berge de droite.

Q18. Écrire une fonction de signature `mouvement_chemin : chemin_caillou -> chemin_caillou list` qui, en fonction de l'état du chemin, calcule l'état suivant après les opérations suivantes (si elles sont permises) :

- (i). déplacement d'un randonneur venant de la berge de gauche,
- (ii). déplacement d'un randonneur venant de la berge de droite,
- (iii). saut d'un randonneur venant de la berge de gauche,
- (iv). saut d'un randonneur venant de la berge de droite.

On donne la syntaxe OCaml pour créer une liste de N entiers $i : \text{List.init } N \text{ (fun } x \rightarrow i)$.
Par exemple, `List.init 5 (fun x -> 2);;` renvoie `[2; 2; 2; 2; 2]`.

Q19. Écrire une fonction de signature `passage : int -> int`, utilisant la question précédente, telle que l'appel `passage nG nD` résout le problème de passage de nG randonneurs venant de la berge de gauche et nD randonneurs venant de la berge de droite. Par exemple, `passage 3 2` permet de passer de `[1; 1; 1; 0; 2; 2]` à `[2; 2; 0; 1; 1; 1]`.

Partie III - Calcul d'une coupe minimum d'un graphe

Un agent secret a pour mission de perturber le réseau de communications ennemi en coupant certains fils dans le réseau. Ayant peu de moyens, l'agent a pour consigne de couper le moins de fils possibles pour accomplir cette tâche. Le réseau de communications est modélisé à l'aide d'un *multigraphe* non orienté $G = (S, A)$, où S est l'ensemble des sommets du graphe et A l'ensemble de ses arêtes. Résoudre le problème de l'agent secret, c'est rechercher une *coupe minimum* dans G .

Définition 4 (Multigraphe)

Un multigraphe est un graphe dans lequel des couples de mêmes sommets peuvent être reliés par plus d'une arête.

Dans toute la suite, on considérera les multigraphes sans arêtes du type (s, s) (boucles).

Définition 5 (Coupe)

Une coupe d'un multigraphe non orienté $G = (S, A)$ est une partition non triviale (S_1, S_2) de S , c'est-à-dire telle que $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$ et $S_1, S_2 \neq \emptyset$. On dira que les arêtes reliant S_1 à S_2 sont les arêtes de la coupe.

La *taille* d'une coupe (S_1, S_2) est définie par $|(S_1, S_2)| = |\{(s, t) \in A, s \in S_1, t \in S_2\}|$. Autrement dit, c'est le nombre d'arêtes de G qui relient S_1 et S_2 .

Définition 6 (Coupe minimum)

Une coupe minimum est une coupe de taille minimale.

Pour trouver une coupe minimum d'un multigraphe $G = (S, A)$, on pourrait énumérer l'ensemble des coupes possibles (il y en a $2^{|S|}$...), ou encore utiliser un algorithme de flot (le plus efficace étant en $O(|S|^3)$). Nous proposons dans la suite un algorithme probabiliste, l'algorithme de Karger, basé sur la notion de contraction d'arête.

III.1 - Contraction d'arête

Soient $G = (S, A)$ un multigraphe et $a = (s, t) \in A$ une arête de G de sommets s et t . *Contracter* l'arête a consiste à :

- (i). créer un nouveau sommet $u = (st)$ dans S . u est un *supersommet* du nouveau graphe,
- (ii). pour toute arête (r, s) ou (r, t) , $r \in S$, ajouter une arête (r, u) à A . Dans le cas où (r, s) et (r, t) existent, deux arêtes reliant r à u sont créées.
- (iii). Supprimer de A toutes les arêtes ayant s ou t comme extrémité.
- (iv). Supprimer s et t de S .

Un supersommet peut donc être vu comme un ensemble de deux sommets du graphe initial, obtenu après une contraction d'arête.

Le multigraphe obtenu est appelé *graphe contracté* et est noté G/st . Si plusieurs arêtes relient s à t dans G , contracter l'une d'entre elles supprime toutes les autres du graphe G/st . Dans toute la suite, on notera G_0 le graphe initial, avant transformations par contractions.

On considère le graphe G_0 de la **figure 2**.

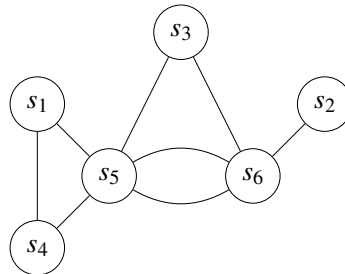


Figure 2 - Graphe G_0 exemple

Q20. Donner le graphe obtenu par contraction d'une arête (s_5, s_6) .

Après plusieurs contractions, un supersommet u du graphe résultant contient un sous-ensemble $S_u \subseteq S$ de sommets de G .

Q21. Soit u un supersommet et $s, t \in S_u$. Montrer qu'il existe un chemin Γ entre s et t dans G_0 tel que chaque arête de Γ a été contractée.

Q22. Montrer qu'en contractant une arête s, t dans un multigraphe G , la taille d'une coupe minimum du graphe contracté G/st est au moins égale à la taille d'une coupe minimum de G .

Q23. Montrer que la taille d'une coupe minimum de G/st est strictement plus grande que celle d'une coupe minimum de G si et seulement si l'arête (s, t) est une arête de toutes les coupes minimum de G .

III.2 - Premier algorithme

On construit un premier algorithme qui essaye de trouver une coupe minimum en contractant aléatoirement une arête de G , jusqu'à ce qu'il ne reste que deux sommets (**algorithme 3**).

Algorithme 3 - Algorithme de Karger.

Karger(G, n)

Entrées : $G = (S, A)$ multigraphe non orienté, $n \in \mathbb{N}$

Sorties : Une coupe minimum de G .

début

pour $i = |S|$ à n **en décrémentant de 1 faire**

 Tirer aléatoirement (loi uniforme) une arête $a = (s, t) \in A$

$G = G/st$

// Appel

Karger($G, 2$)

Q24. Appliquer l'**algorithme 3** au graphe de la **figure 2**, en contractant successivement (s_5, s_6) , $((s_5 s_6), s_2)$, (s_1, s_4) et $((s_5 s_6 s_2), s_3)$. Chaque graphe contracté sera représenté. La coupe calculée est-elle une coupe minimum ?

D'après la **Q23**, tant que l'on ne contracte pas une arête faisant partie de toutes les coupes minimum de G , alors l'**algorithme 3** trouvera une coupe minimum. On cherche alors la probabilité de ne jamais contracter une telle arête.

Q25. Soit $d(s)$ le degré d'un sommet $s \in S$. Montrer que $\sum_{s \in S} d(s) = 2|A|$ (lemme des poignées de main).

Q26. En déduire qu'une coupe minimum a une taille d'au plus $2|A|/|S|$.

Soit $C = (S_1, S_2)$ une coupe minimum de G . L'objet des questions **Q27** et **Q28** est de minorer la probabilité que l'**algorithme 3** renvoie C . Pour cela, on remarque que cet algorithme ne renvoie pas C si et seulement si lors d'une itération on choisit une arête qui traverse C , c'est-à-dire telle qu'un sommet est dans S_1 et l'autre dans S_2 .

Q27. Quelle est la probabilité maximum de choisir une arête qui traverse C ?

Q28. En déduire que la probabilité $P_{|S|}$ que l'**algorithme 3** renvoie C est supérieure ou égale à $\frac{2}{|S|(|S| - 1)}$.

Q29. Déduire de la question précédente qu'un multigraphe non orienté $G = (S, A)$ possède au plus $\frac{|S|(|S| - 1)}{2}$ coupes minimum.

III.3 - Deuxième algorithme

Le résultat précédent n'est pas satisfaisant d'un point de vue complexité. On peut cependant l'améliorer facilement en utilisant une technique d'*amplification* : on itère l'**algorithme 3** plusieurs fois et on retourne la valeur minimale obtenue (**algorithme 4**).

Algorithme 4 - Algorithme de Karger amplifié.

KargerAmplifie(G, N)

Entrées : $G = (S, A)$ multigraphe non orienté, $N \in \mathbb{N}$.

Sorties : Une coupe minimum de G .

début

```

     $t = \infty$ 
    pour  $i = 1$  à  $N$  faire
         $X = \text{Karger}(G, 2)$ 
        si  $|X| < t$  alors
             $t = |X|$ 
             $C = X$ 
    retourner  $C$ 

```

Q30. Montrer que la probabilité maximale que l'**algorithme 4** ne trouve pas une coupe minimum est égale à $\left(1 - \frac{2}{|S|(|S| - 1)}\right)^N$.

Q31. On rappelle que pour tout $x \in \mathbb{R}$, $1 + x \leq e^x$. Montrer que la probabilité que l'**algorithme 4** renvoie une coupe minimum est supérieure à $1 - e^{2N/|S|(|S|-1)}$. En déduire, pour $c > 0$ fixé, la plus petite valeur de N telle que cette probabilité soit supérieure à $1 - \frac{1}{N^c}$.

Q32. Les algorithmes proposés dans cette partie sont des algorithmes probabilistes. Sont-ils de type Monte Carlo ou Las Vegas ? Justifier la réponse.

III.4 - Implémentation en langage C

Cette partie comporte des questions de programmation qui seront abordées en utilisant **exclusivement le langage C**. Les codes seront commentés de manière pertinente.

On suppose qu'un graphe est codé à l'aide de l'ensemble de ses arêtes, chaque arête étant définie par ses deux sommets, représentés par des entiers.

Q33. Définir en C des types structurés *Arete* et *Graphe*. Pour ce dernier, on s'attachera à avoir un accès direct au nombre de sommets et au nombre d'arêtes. On rappelle la définition d'un type structuré par `struct nom_s {type1 champ1;...;typen champn;} et ensuite typedef struct nom_s nom;.`

Pour tout u supersommet, les S_u forment une partition de S . Pour coder ces sous-ensembles, on a donc naturellement recours à la structure de données Unir & Trouver (Union-Find), qui va permettre de trouver rapidement à quel sous-ensemble S_u un sommet $s \in S$ appartient (Trouver) et de fusionner deux supersommets S_u et S_v déjà construits (Unir).

On suppose donc disposer :

- (i). d'un type `subset`, décrivant un supersommet d'un graphe contracté.

```
struct subset
{
    int parent; // recherche par compression de chemin pour Trouver
    int rang;   // Union par rang.
};
typedef struct subset subset;
```

- (ii). d'une fonction de prototype `int Trouver(subset subsets[], int s)`, qui retourne le numéro du supersommet dans lequel le sommet s se trouve (par compression de chemin),
- (iii). d'une fonction de prototype `void Unir(subset subsets[], int Su, int Sv)` qui fusionne les deux supersommets S_u et S_v (union par rang) et maintient à jour la liste des supersommets.

Il n'est pas demandé d'écrire ces deux dernières fonctions.

- Q34.** Écrire une fonction de prototype `int contracteArete(Graphe G, subset subsets[], Arete a)` qui contracte l'arête a . On veillera à ne pas contracter l'arête si ses deux extrémités sont dans le même supersommet. La fonction renvoie 0 si aucune arête est contractée et -1 sinon.
- Q35.** Écrire une fonction de prototype `int compteAretesCoupe(Graphe G, subset subsets[])` qui compte le nombre d'arêtes du graphe contracté final G résultant de l'**algorithme 3**. `subsets` est la partition des sommets du graphe initial S calculée par l'**algorithme 3**.
- Q36.** En déduire une fonction de prototype `int kargerMinCut(Graphe G0)` qui renvoie la taille de la coupe calculée par l'**algorithme 3** sur le graphe G_0 . Les supersommets initiaux sont les $|S|$ sommets de G_0 , chacun ayant un rang nul et un numéro parent égal au numéro du sommet. Pour effectuer un tirage aléatoire uniforme, on utilisera la fonction `int rand()`; de la librairie `stdlib.h` qui renvoie un nombre aléatoire entre 0 et `RAND_MAX` ≥ 32767 . `RAND_MAX` est une constante définie dans `stdlib.h`.
- Q37.** Donner la complexité au pire des cas de cet algorithme.

FIN